Machine Learning for Time Series

Lecture 2: Feature Extraction and Selection

Laurent Oudre
laurent.oudre@ens-paris-saclay.fr

Master MVA
2026-2027

Contents

1. Problem statement

2. Feature Extraction
2.1 Stationarity and ergodicity
2.2 Statistical features
2.3 Spectral features
2.4 Local symbolic features
2.5 Information theory features
2.6 Convolutional features
2.7 Deep Learning features
2.8 Other features

3. Feature Selection
3.1 Unsupervised setting
3.2 Supervised setting

4. Conclusion

Contents

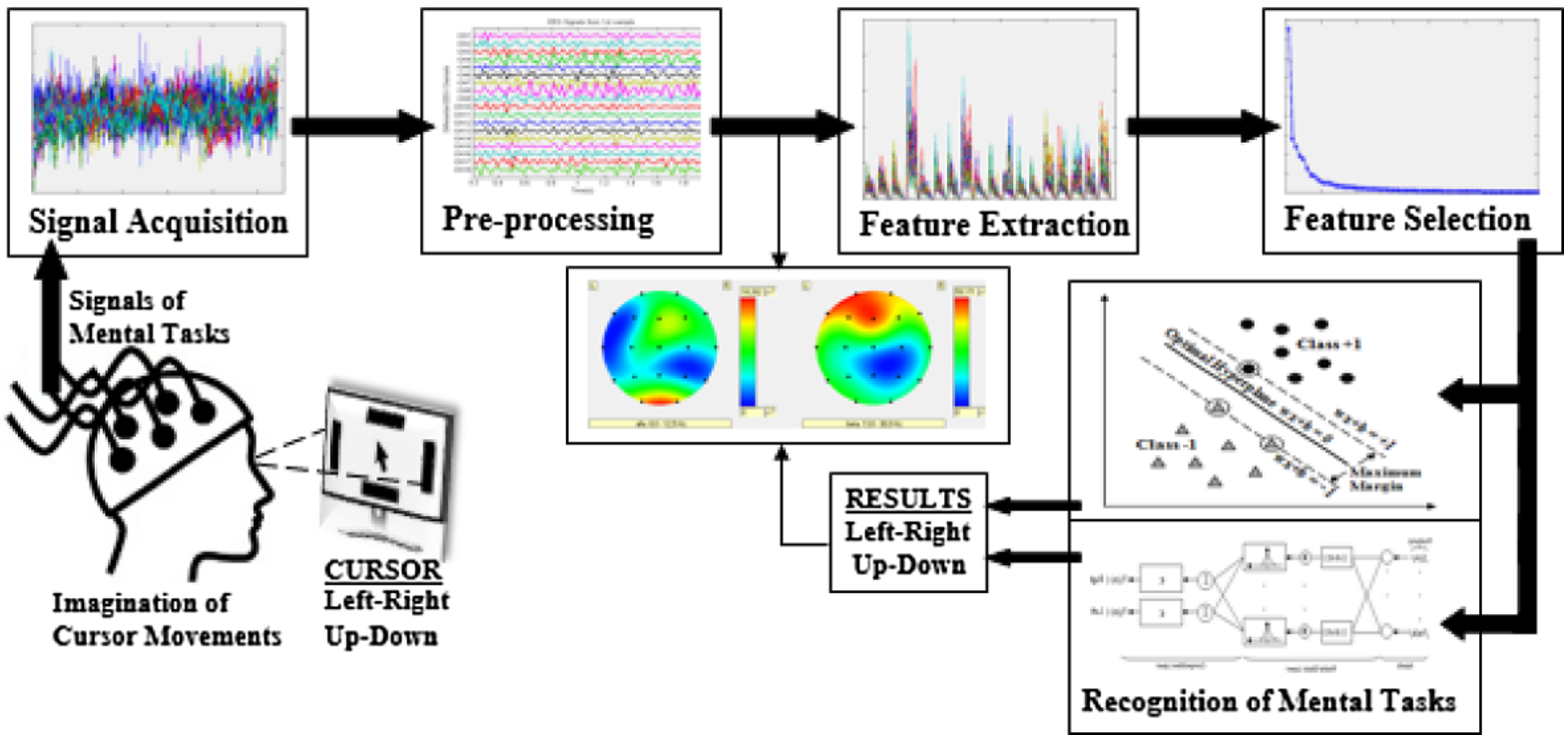
1. Problem statement

2. Feature Extraction

3. Feature Selection

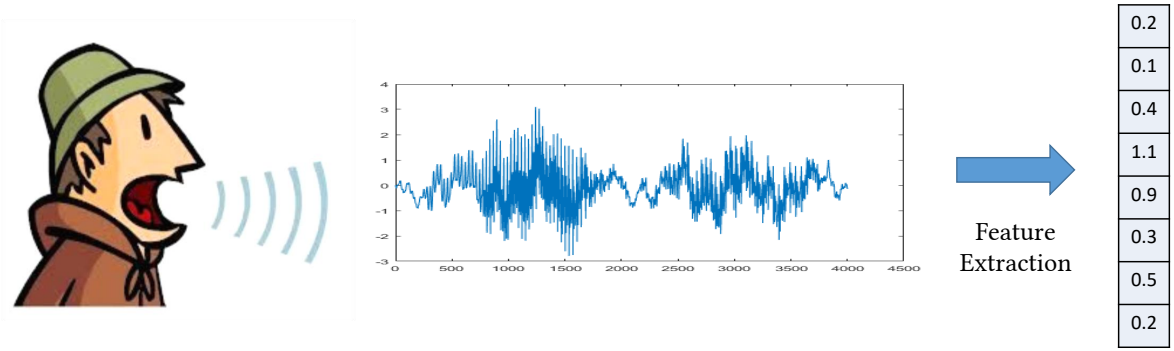
4. Conclusion

Motivation



Feature extraction/selection: fundamental step for using ML on time series

Problem 1: Feature Extraction

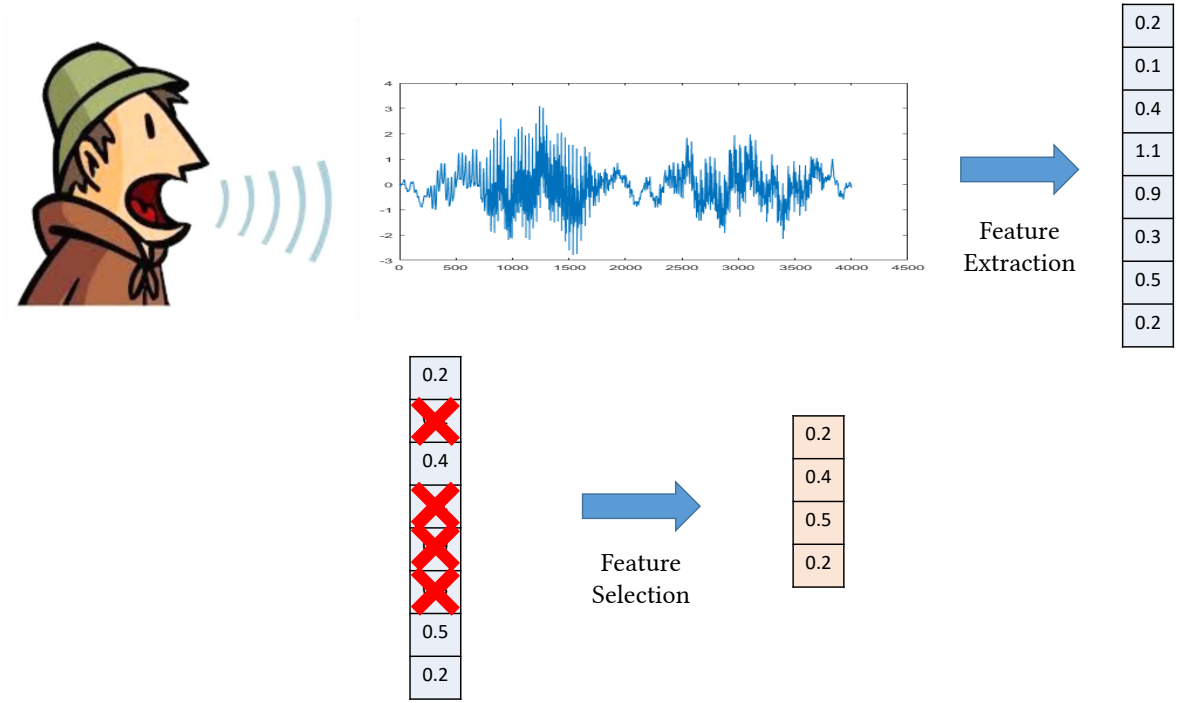


Feature Extraction

Given a time series of length N, extract a set of D features that characterizes the time series

- Useful for indexing, classification, clustering...
- Often **based on expertise** : feature engineering (construction of ad hoc features from domain knowledge)
- **Bag of features** representation for time series

Problem 2: Feature Selection



Feature Selection

Given a dataset of M time series represented each with D features, select the $K < D$ features that are relevant for a given task

- Can be performed in a **supervised** (labeled data) or **unsupervised** setting

Contents

1. Problem statement

2. Feature Extraction
2.1 Stationarity and ergodicity
2.2 Statistical features
2.3 Spectral features
2.4 Local symbolic features
2.5 Information theory features
2.6 Convolutional features
2.7 Deep Learning features
2.8 Other features

3. Feature Selection

4. Conclusion

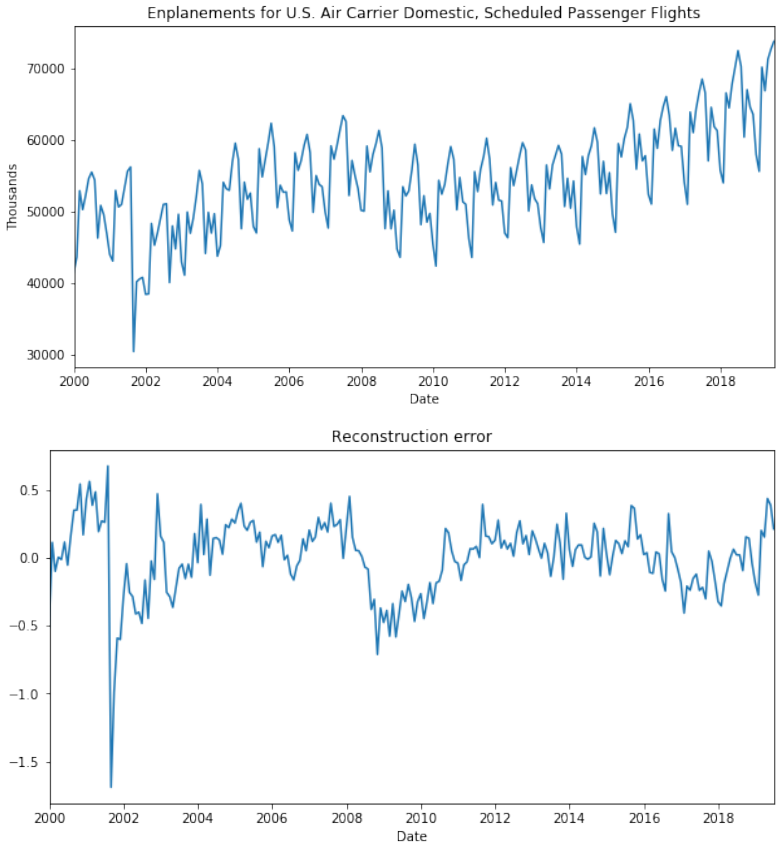
Good feature engineering

- **Good feature engineering is based on domain knowledge**: adapted to the data and to the underlying model (see Lecture 3 on Models and Representation Learning)
- In the general case, one solution is to use classical features that capture statistical or physical properties of the time series
- **Reminder** : two visions
  - **Physics :** a time series $x[0 : N-1]$ is the result of the digitization of a physical phenomenon $x(t)$. Physical properties of this phenomenon can be retrieved and analyzed through the study of $x[0 : N-1]$ (and vice versa).

    Spectral features
  - **Statistics :** a time series $x[0 : N-1]$ is a realization of a stochastic process $X[0 : N-1]$. Statistical properties of this phenomenon can be retrieved and analyzed through the study of $x[0 : N-1]$ (and vice versa).

    Statistical features

The hidden assumptions behind feature extraction



Would it make sense to compute the empirical mean of these time series ?

- Some data scientists extract bags of features without thinking whether they are relevant/correct or not
- Even a simple feature such as the empirical mean can be wrong
- Before attempting to extract relevant information from a time series, one should ask the question: **is it meaningful to have one unique feature to describe the whole time series?**

Time series are random processes

- We will start with the **statistical point of view**
- To formalize this question, we shall assume that $x[0 : N-1]$ corresponds to a realization of a stochastic process $X[0 : N-1]$
- Each $X[n]$ can be seen as a random variable
- Our capacity to extract meaningful features depends on the **statistical properties of the implicit random process** $X[0 : N-1]$

Two fundamental properties: stationarity

- **Stationarity : The statistical properties of the time series do not change over time**
  - **Order 1**

$$\forall n, \quad \mathbb{E}\left[X[n]\right] = \mu$$

  - **Order 2**

$$\forall n_1, n_2, \quad \mathbb{E}\left[X[n_1]X[n_2]\right] = \gamma_X[|n_2 - n_1|]$$

  - Order 1 + Order 2 $\rightarrow$ **wide-sense stationarity** (most common assumption)

Two fundamental properties: ergodicity

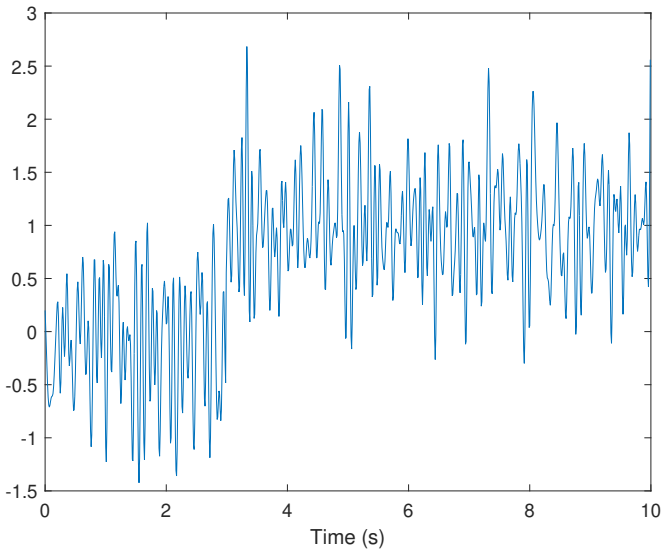
- **Ergodicity: Statistical properties can be retrieved from temporal properties.** Assuming that the time series is **wide-sense stationary**:
  - **Order 1**

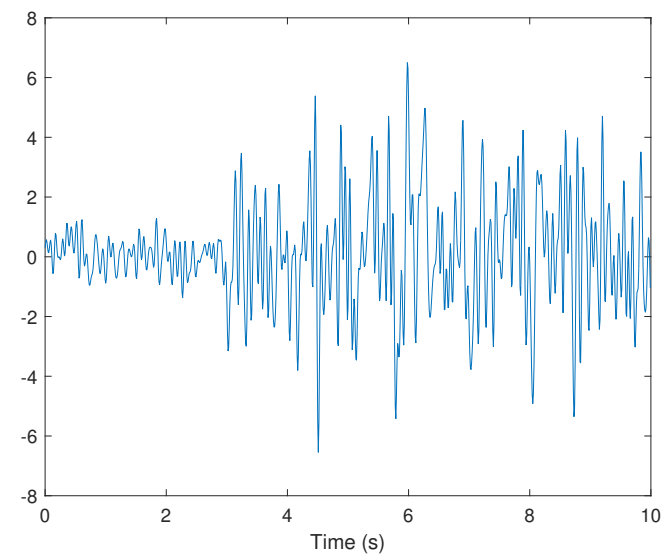
$$\forall n, \quad \mathbb{E}\left[X[n]\right] = \mu = \lim_{N\to\infty} \frac{1}{N} \sum_{n=0}^{N-1} x[n]$$

  - **Order 2**

$$\forall n, k, \quad \mathbb{E}\left[X[n]X[n+k]\right] = \gamma_X[k] = \lim_{N\to\infty} \frac{1}{N} \sum_{n=0}^{N-1} x[n]x[n+k]$$

Stationarity vs. non-stationarity





- None of these signals are stationary: any statistical features computed on the whole time series will be **wrong**
- In order to prevent this to happen, **two solutions** exist
  - Divide the signals into small frames where the signal is assumed to be stationary and ergodic
  - Use a change-point detection algorithm to detect these changes and work separately on each segment (see Lecture 5)

Statistical features

- A popular choice for features consists in extracting statistical properties of the time series
- In this context, we assume that $x[0 : N-1]$ corresponds to a realization of a stochastic process $X[0 : N-1]$, that is **ergodic and wide-sense stationary**
- Statistical features of $X[0 : N-1]$ can be retrieved from estimates based on $x[0 : N-1]$

Statistical features

Assuming that $X[0 : N-1]$ is ergodic and wide-sense stationary, we can estimate several statistical properties from $x[0 : N-1]$

- Mean

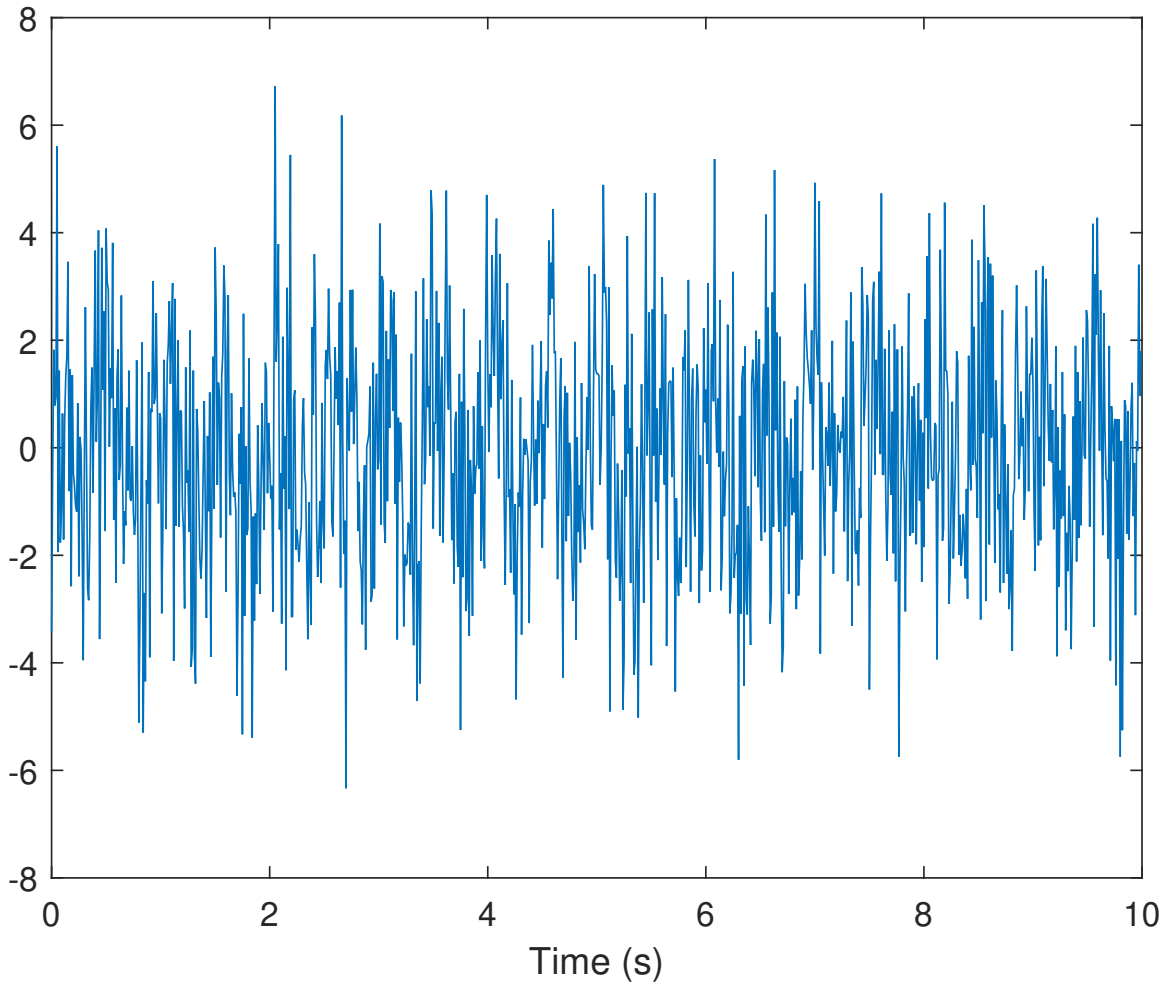
$$\hat{\mu} = \frac{1}{N} \sum_{n=0}^{N-1} x[n]$$

- Autocorrelation function

$$\hat{\gamma}_x^{biased}[m] = \frac{1}{N} \sum_{n=0}^{N-1} x[n]x[n+m] \quad \text{where } x[n] = 0 \text{ for } n \notin [\![0, N-1]\!]$$

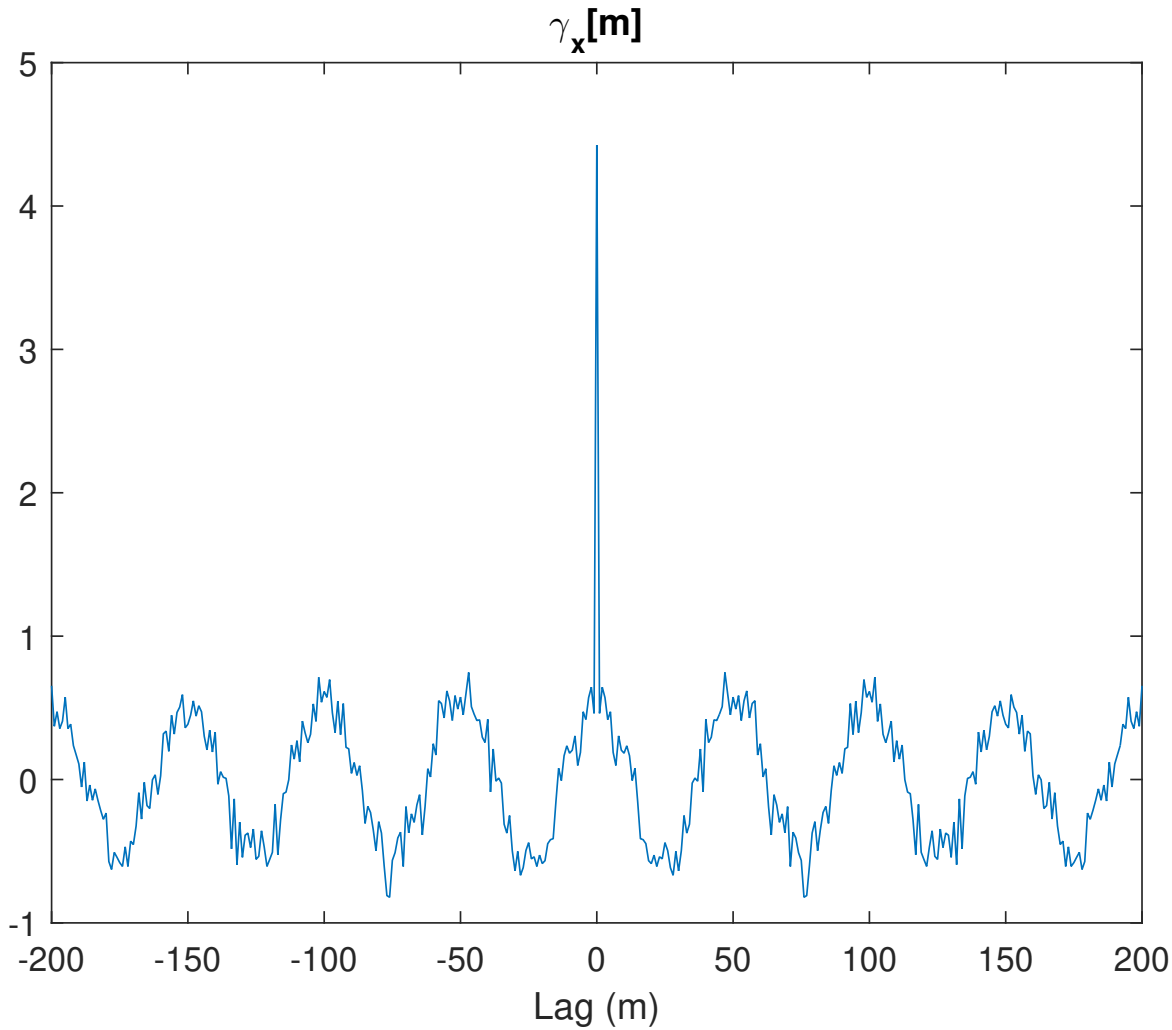
$$\hat{\gamma}_x^{unbiased}[m] = \frac{1}{N-|m|} \sum_{n=0}^{N-1} x[n]x[n+m] \quad \text{where } x[n] = 0 \text{ for } n \notin [\![0, N-1]\!]$$

How to use the autocorrelation function



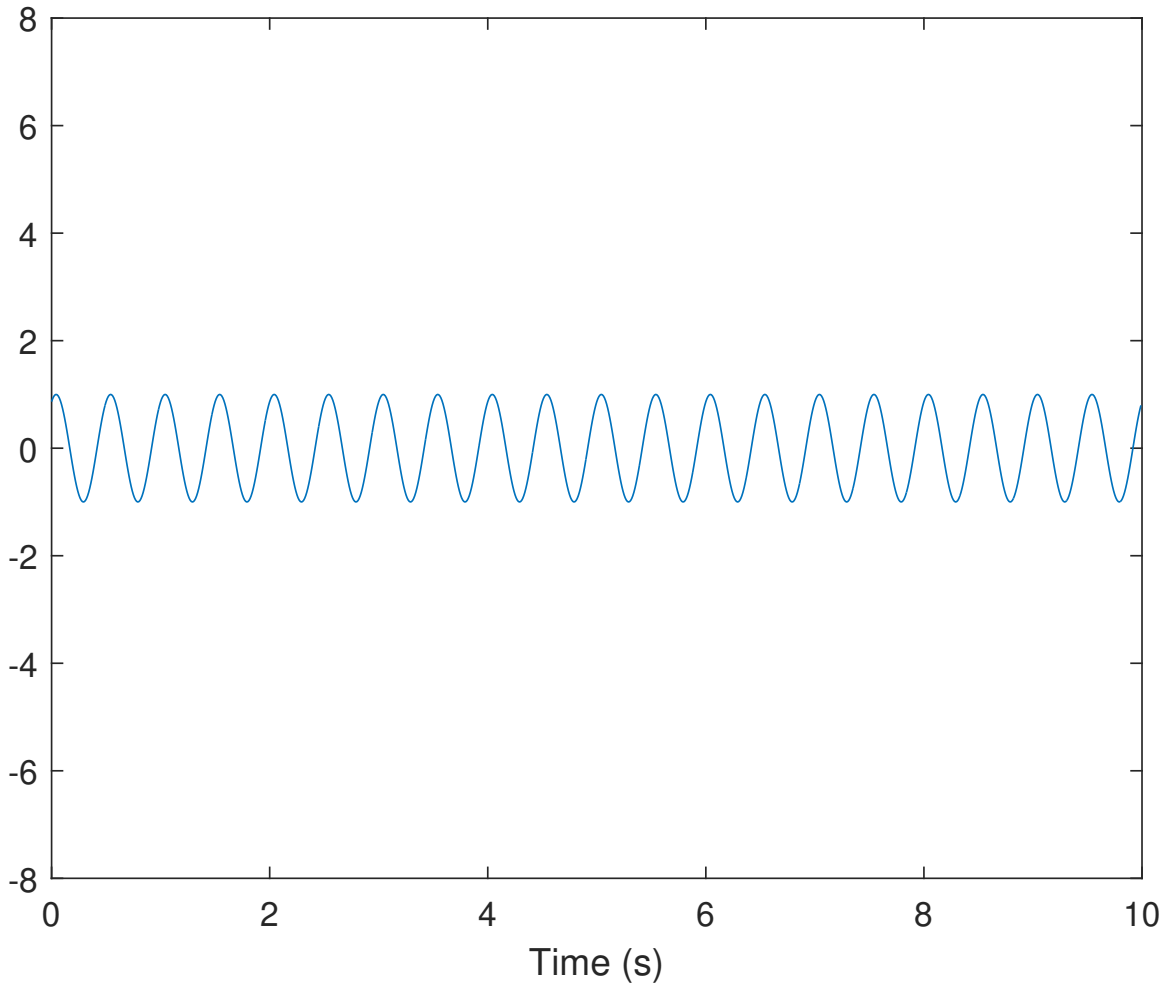
Original signal, sampling frequency 100 Hz

How to use the autocorrelation function



Autocorrelation function, peaks are visible for lags multiple of 50

How to use the autocorrelation function



A periodic signal with period $50 \times \frac{1}{100} = 0.5$ sec was hiding!

Features from autocorrelation function

- Given a lag m, one useful feature for a centered random process is the **renormalized autocorrelation coefficient**

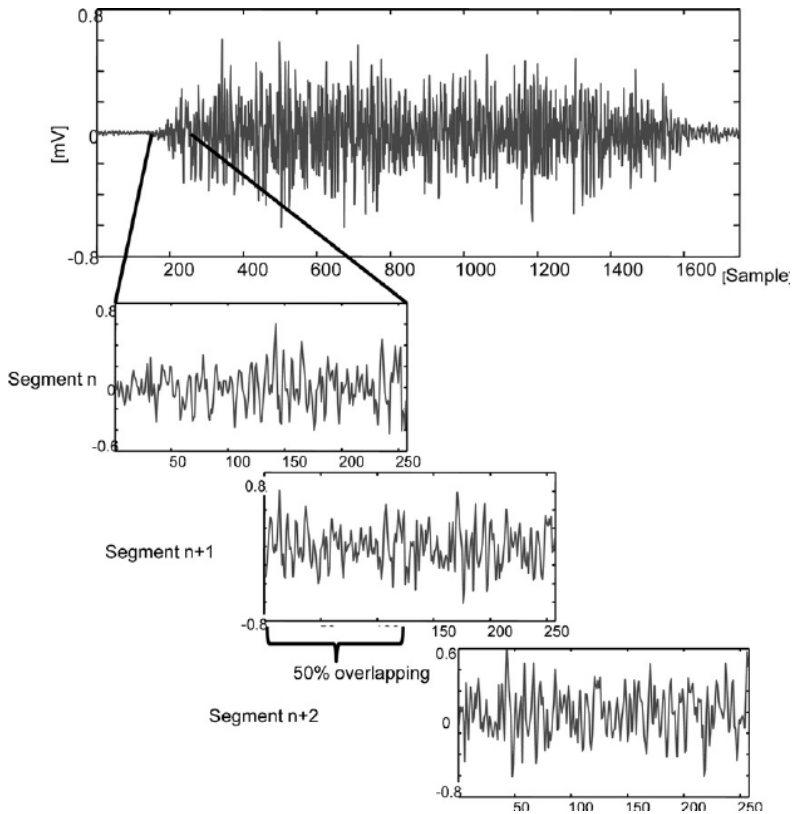
$$\gamma'_X[m] = \frac{\gamma_X[m]}{\gamma_X[0]}$$

that quantifies the correlation between two samples spaced by m in the time series

- These features are very useful when the signal has a seasonality or periodic component (see Lecture 3): in this case, we store $\gamma'_X[m_0]$ where m_0 corresponds to the index of the **first peak** of the autocorrelation function ($m_0 > 0$)

Other statistics

- Other quantities can be estimated such as minimum and maximum values, or **robust statistics** such as median or percentiles (5%, 25%, etc.)
- Most of the time, these purely statistical features remove the time information...
- If necessary, those quantities can be extracted on **sliding windows**, increasing the number of features while preserving the time information



Spectral features

- We now switch to the **physical point of view**
- Several tools derived from Fourier analysis and signal processing can be used to extract relevant features
- In this context, we assume that **x** corresponds to the discrete measurement of a continuous signal $x(t)$
- **Sampling theory:** uniform sampling period T_s and sampling frequency $F_s = \frac{1}{T_s}$

$$x[n] = x(nT_s)$$

Discrete Fourier Transform (DFT)

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi \frac{kn}{N}} \text{ for } 0 \leq k \leq N-1$$

where N is the number of samples

- The space between two observable frequencies is called **frequency resolution**

$$\Delta f = \frac{F_s}{N}$$

- $X[k]$ corresponds to the DFT for the physical frequency

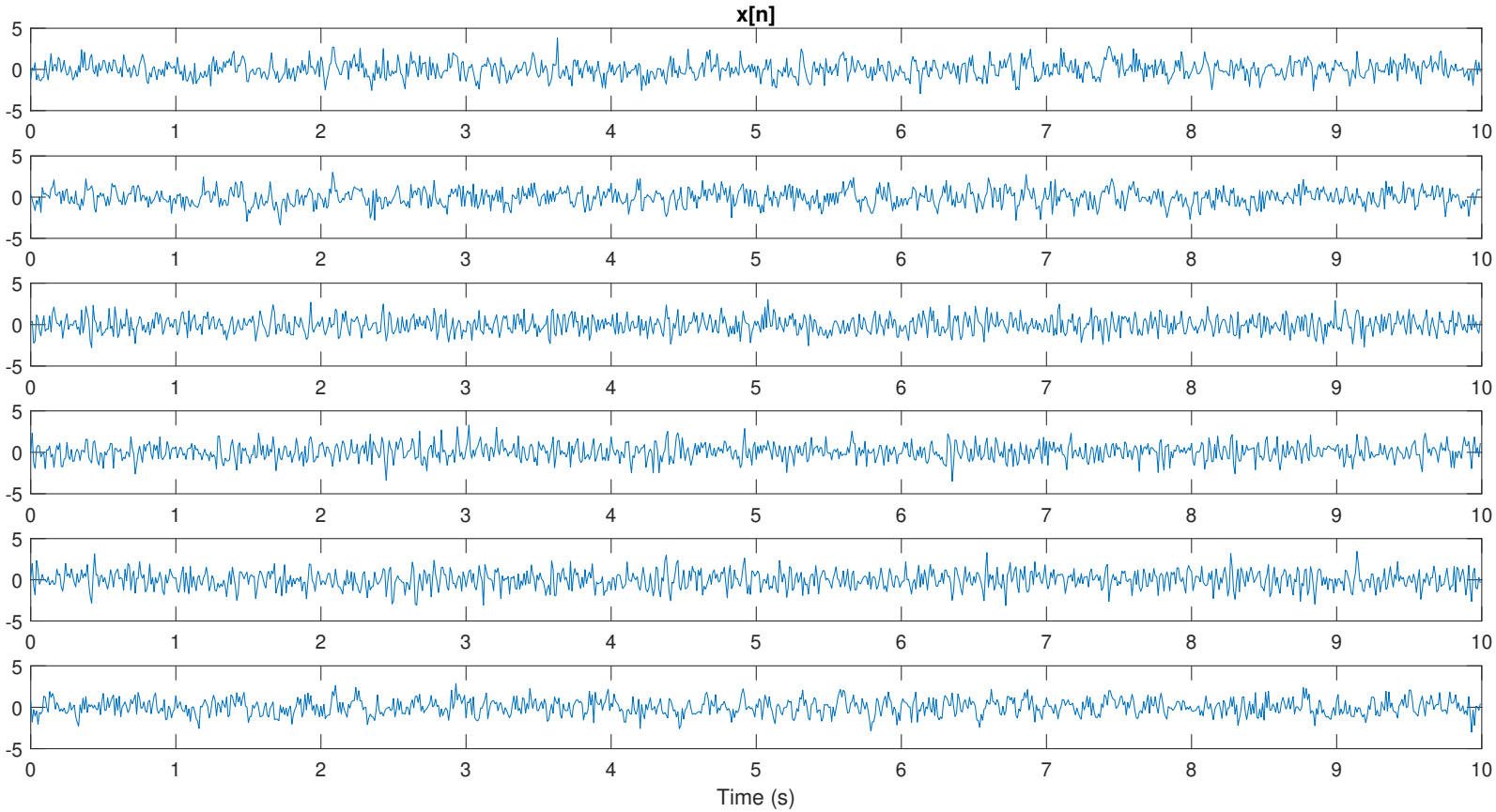
$$f[k] = k\frac{F_s}{N} \text{ for } 0 \leq k \leq \left\lfloor \frac{N}{2} \right\rfloor$$

- By periodicity of the DFT, indices $k > \frac{N}{2}$ correspond to the **negative** frequencies $f[k] = (k-N)\frac{F_s}{N}$
- No physical frequency of modulus greater than $\frac{F_s}{2}$ can therefore be observed (**Nyquist theorem**)

Spectral analysis

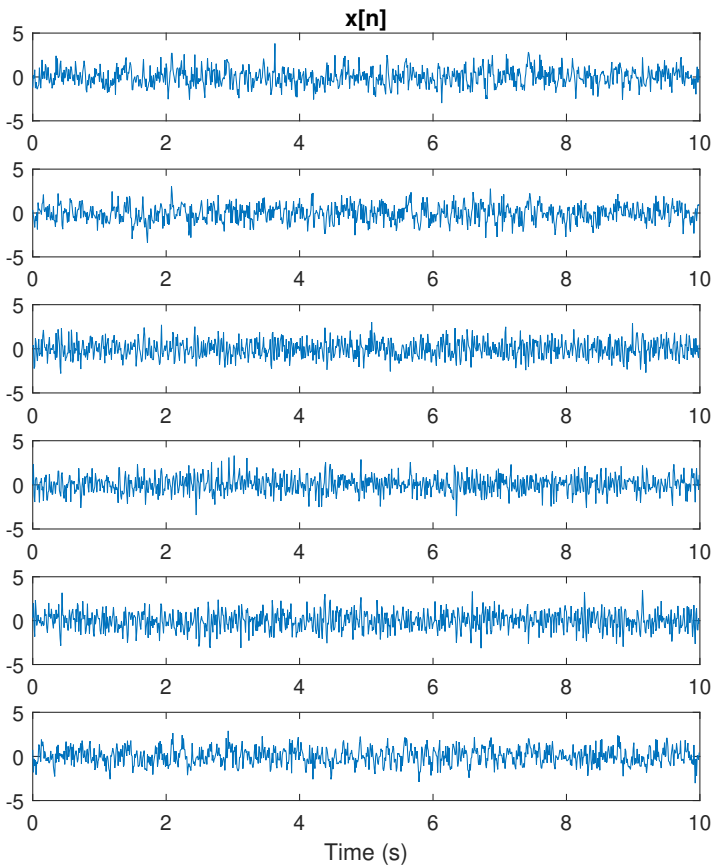
- $X[k]$ is a complex quantity: most of the time, we use the squared absolute values $|X[k]|^2$ instead
- The analysis of the quantity $|X[k]|^2$ can reveal interesting properties of the time series
- $|X[k]|^2$ with low frequencies $f[k]$ correspond to phenomena with **smooth variations**
- $|X[k]|^2$ with large frequencies $f[k]$ correspond to phenomena with **fast variations**
- One very useful plot consists in plotting $|X[k]|^2$ as a function of $f[k]$: such plot is often referred to as **spectrum** (hence spectral analysis)

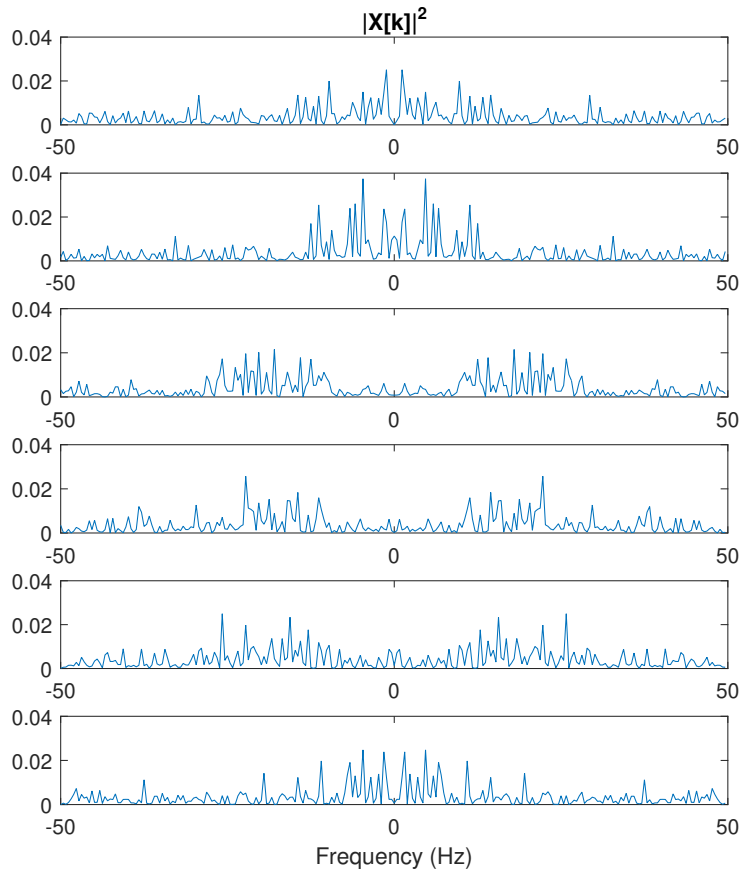
Example



Two classes of signals?

Example





Can be distinguished based on their DFT coefficients

DFT features

- $|X[k]|^2$ coefficients can be useful features
- In most cases, we can merge them on a frequency band of interest $[f_1, f_2]$, with $0 \leq f_1 < f_2 \leq \frac{F_s}{2}$
- The computed quantity is called **relative energy**: it is the energy of the band, normalized by the total energy of the signal

$$E_{[f_1,f_2]} = \frac{\sum_{k, f[k] \in [f_1, f_2]} |X[k]|^2}{\sum_{k, f[k] \in [0, \frac{F_s}{2}]} |X[k]|^2}$$

Other spectral features

Let $P[k] = |X[k]|^2$ and $p[k] = P[k] / \sum_{k'} P[k']$, seen as a probability distribution over the frequency bins. Several features can be computed:

- **Spectral centroid** : $c = \sum_k f[k]\, p[k]$ (where the energy is located)
- **Spectral spread** : $\sigma_s^2 = \sum_k (f[k] - c)^2 p[k]$ (how concentrated it is)
- **Spectral entropy** : $H_s = -\sum_k p[k] \log_2 p[k]$
- **Spectral flatness** : $\text{SFM} = \frac{\left(\prod_k P[k]\right)^{1/K}}{\frac{1}{K}\sum_k P[k]} = \frac{\text{geometric mean}}{\text{arithmetic mean}}$
- **Roll-off** : the frequency below which a fraction β of the energy lies

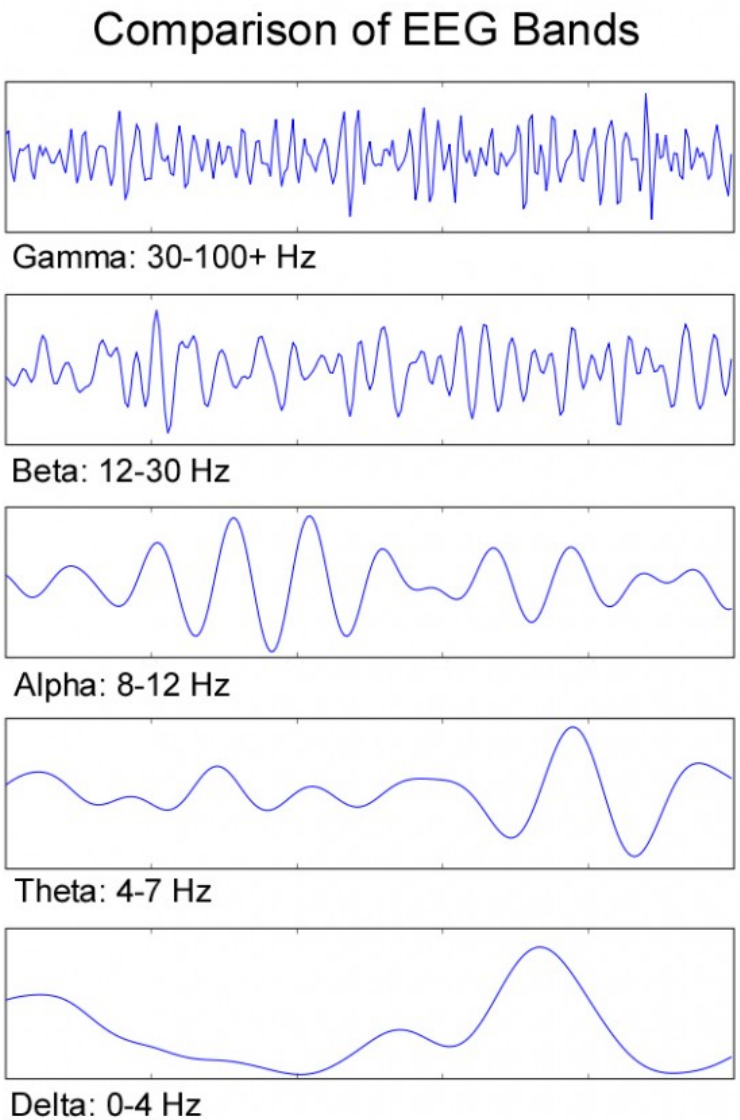
Example: EEG



Electroencephalography (EEG):

- Measures the electrical activity of the brain with a sensor network
- Used to study several neurological diseases (epilepsy, stroke...) and sleep
- Each sleep phase corresponds to the emergence of typical frequencies in the brain

Practical example: EEG



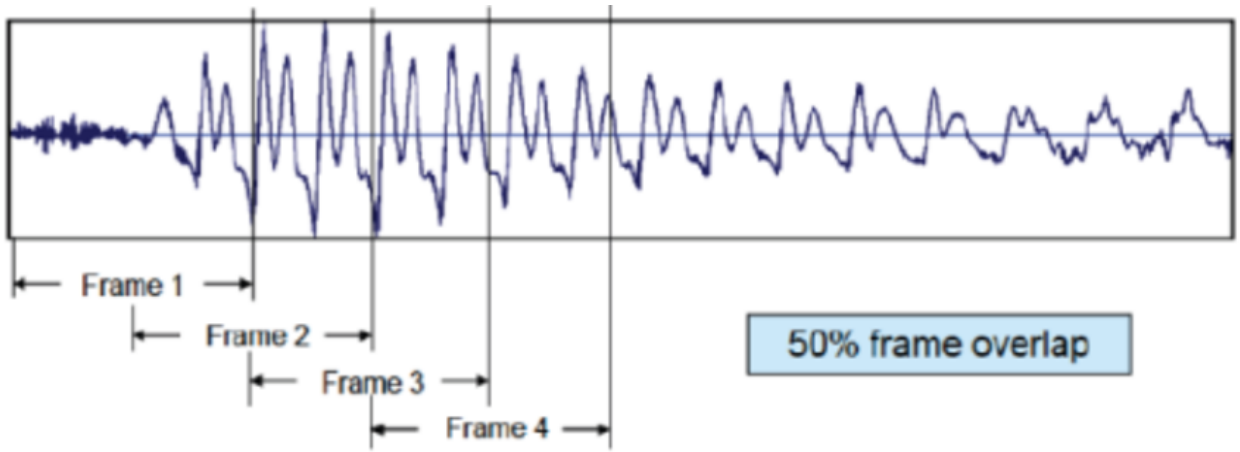
Each sleep phase corresponds to the emergence of typical frequencies in the brain:

- **Delta waves**: deep sleep without dreams
- **Theta waves**: deep relaxation
- **Alpha waves**: light relaxation
- **Beta waves**: awake, active information processing

Spectral features make it possible to determine in which sleep phase a subject is

Spectrogram

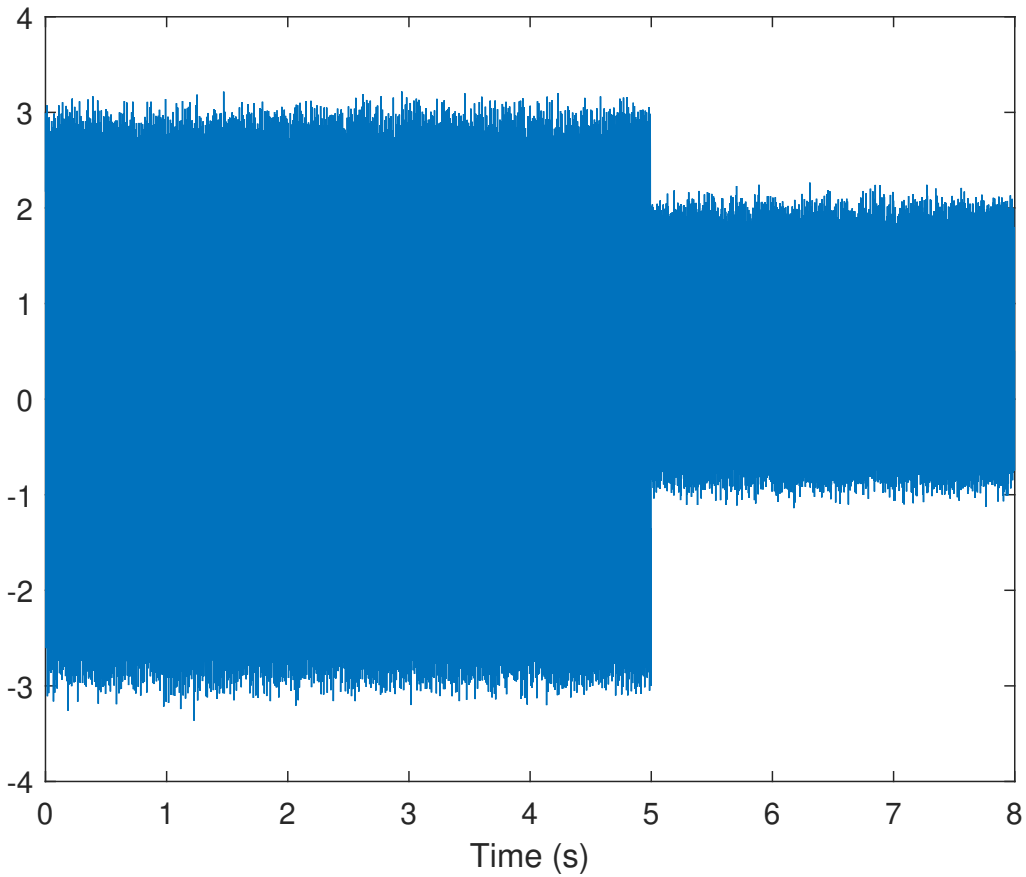
- When the properties of the time series tend to change with time, it is safer to compute the DFT on sliding windows
- By sliding the window along the signal, we recover a time-frequency representation called **spectrogram**



- **Matrix representation**: each column corresponds to the DFT on the window of interest.

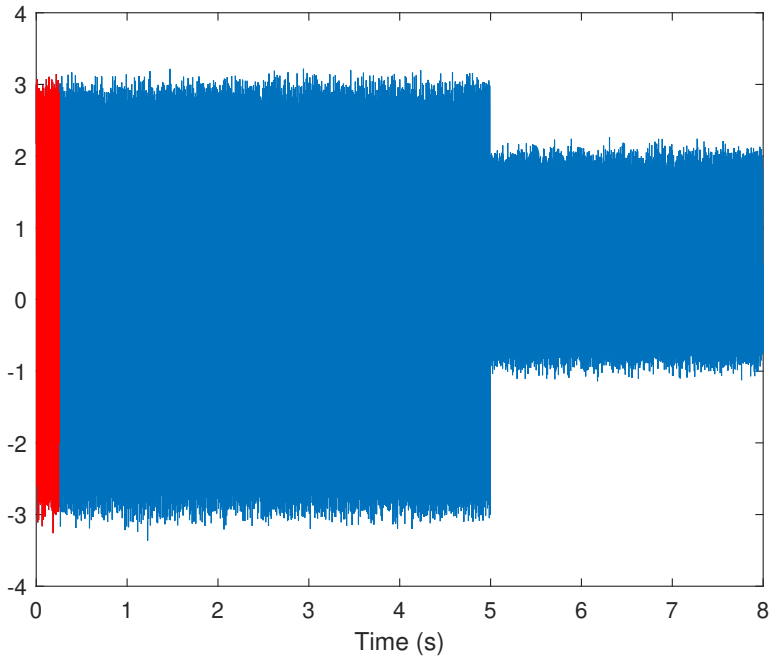
X-axis: frame number, Y-axis: frequency bin

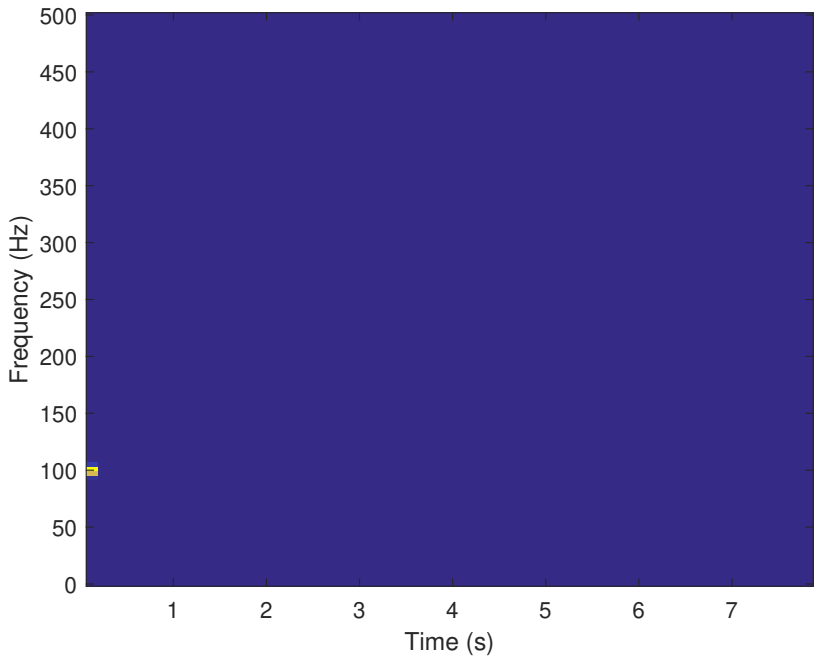
Example



Original signal, F_s = 1000 Hz

Example

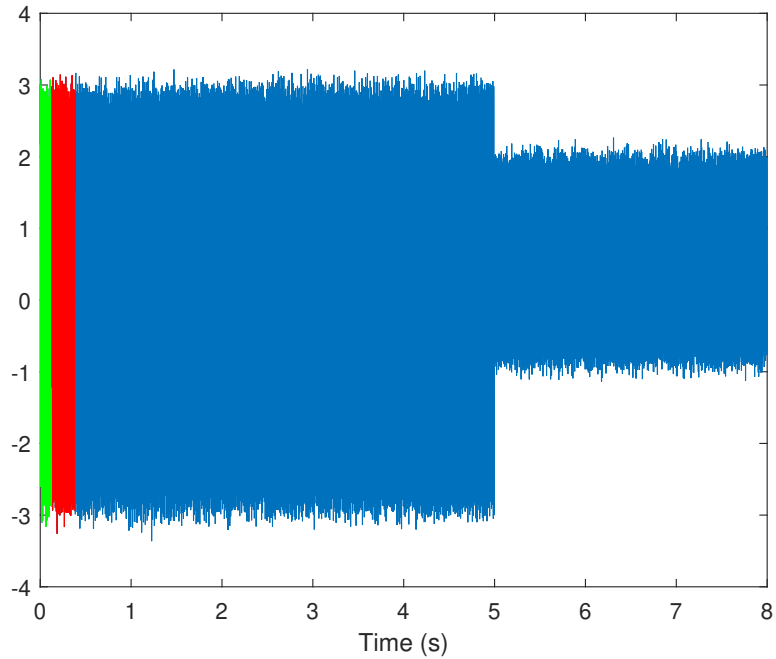


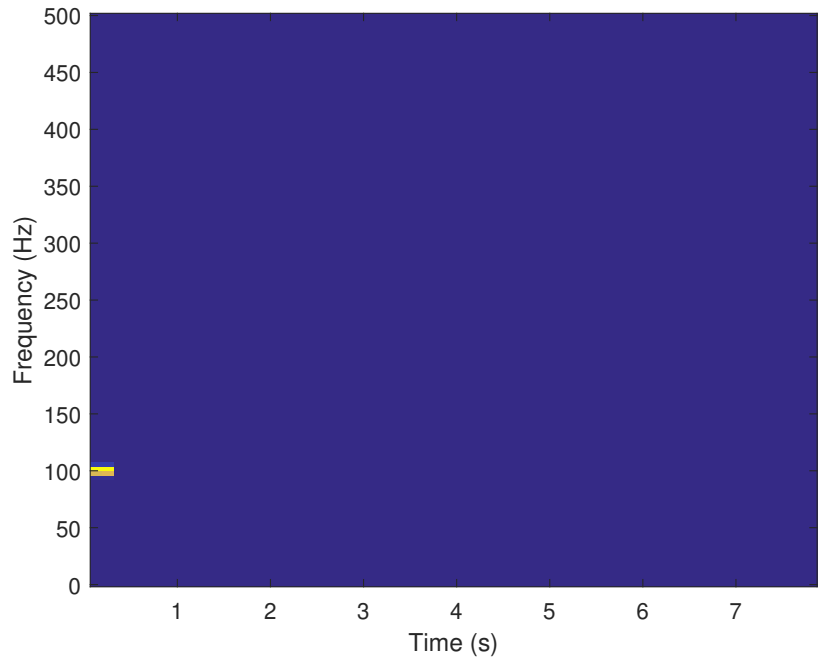


Window length $N_w = 256$

Computation of the DFT on the first frame and storage in the spectrogram matrix...

Example

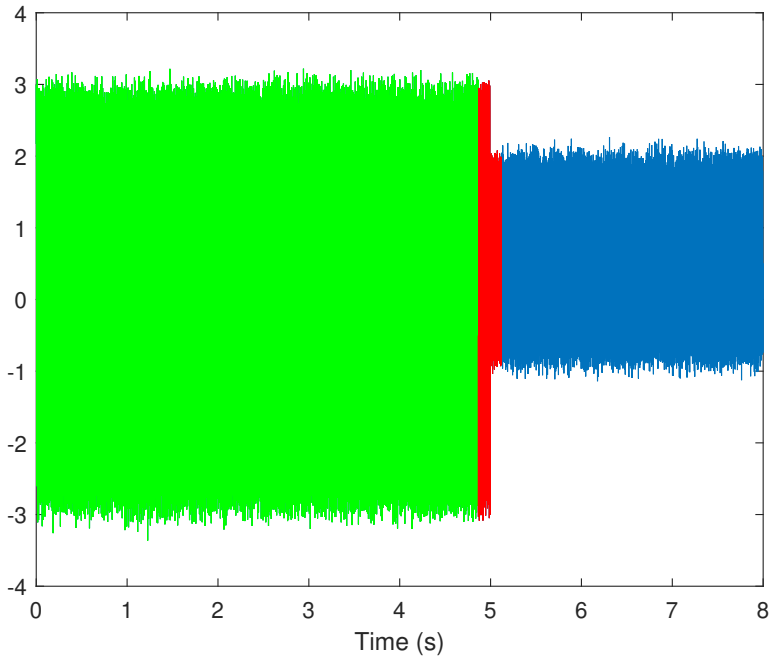


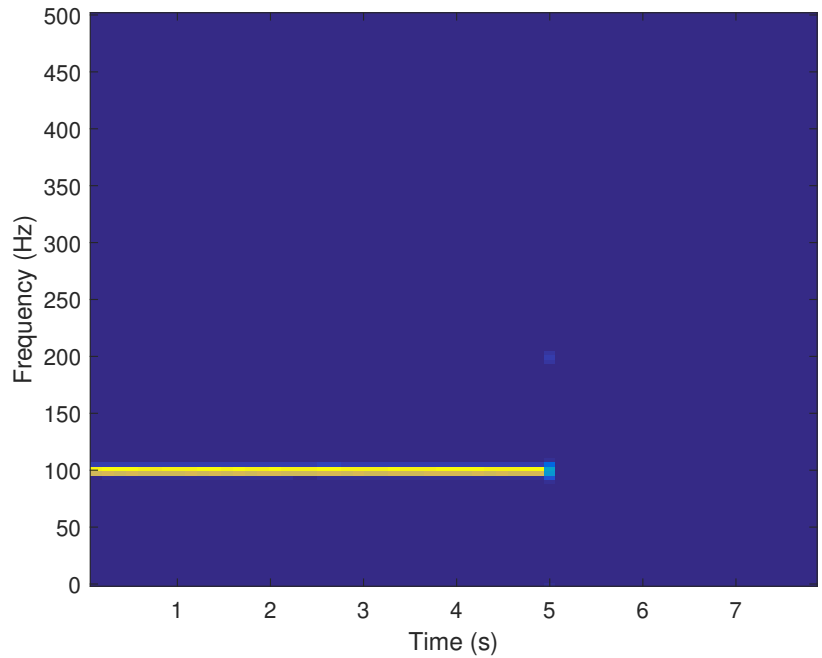


Window length $N_w = 256$

Computation of the DFT on the second frame and storage in the spectrogram matrix...

Example

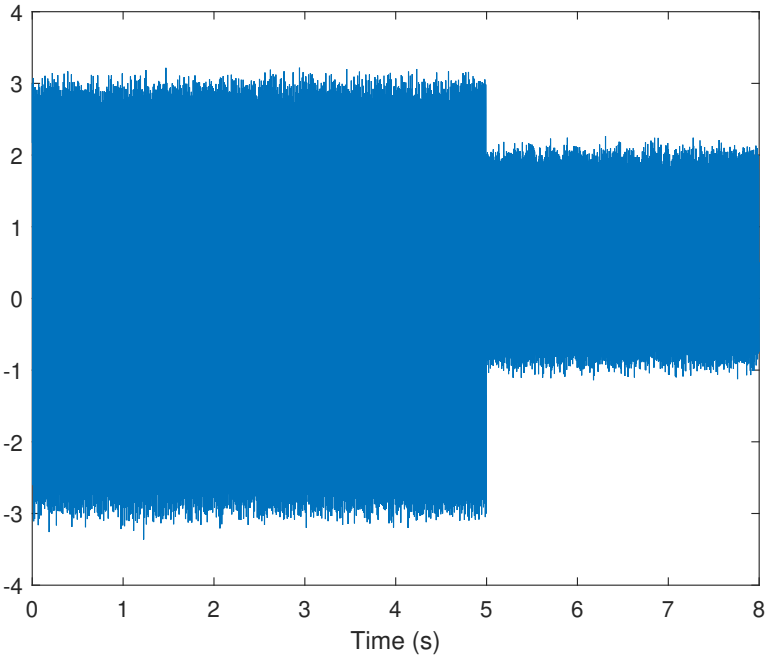


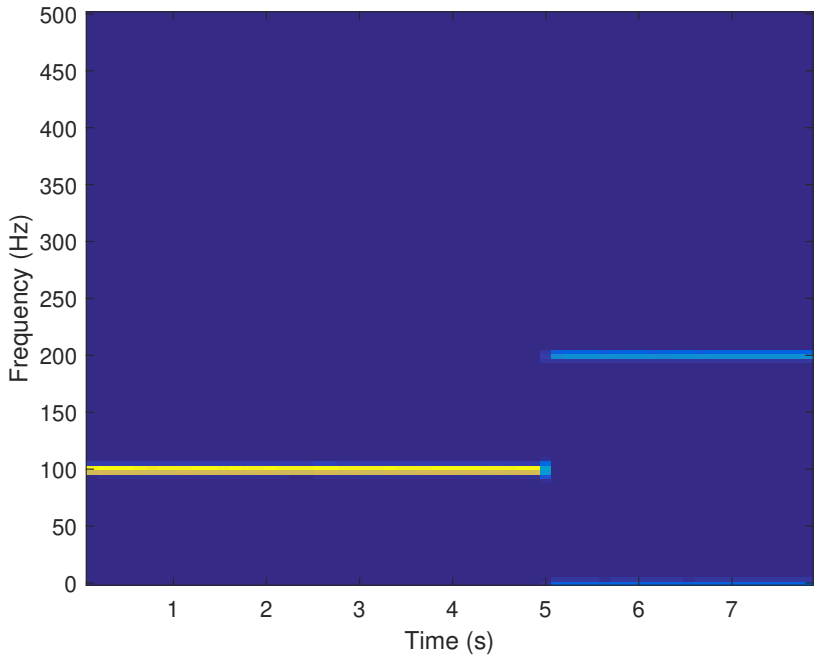


Window length $N_w = 256$

Same process...

Example





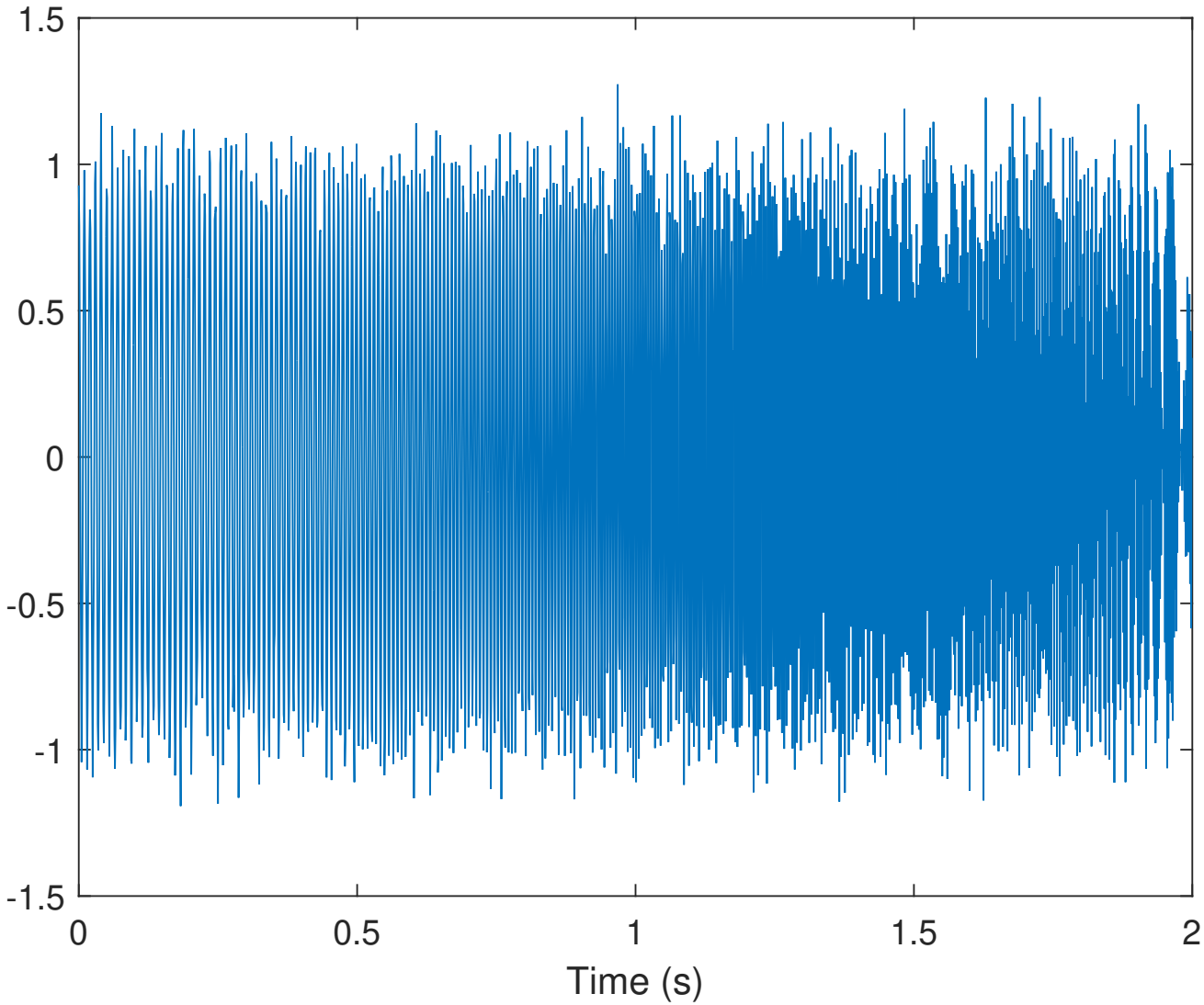
Window length $N_w = 256$

Final result

DFT vs. Spectrogram

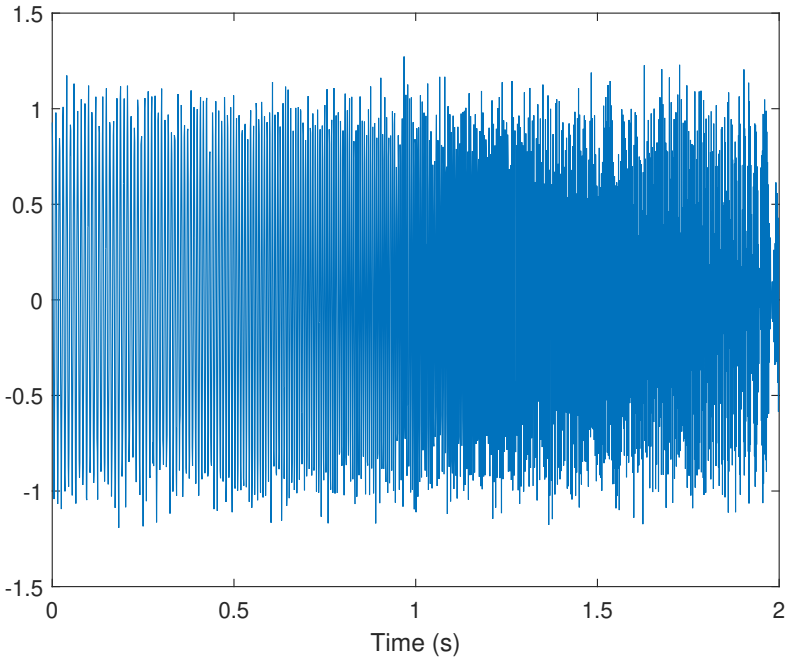
- Only use DFT when you are sure that there are **no abrupt changes** in the time series
- Note that using DFT will tend to average the frequency content on the whole time series, which can be tricky in some application contexts
- For safety, **always first visualize the spectrogram** to make sure that no significant changes occur

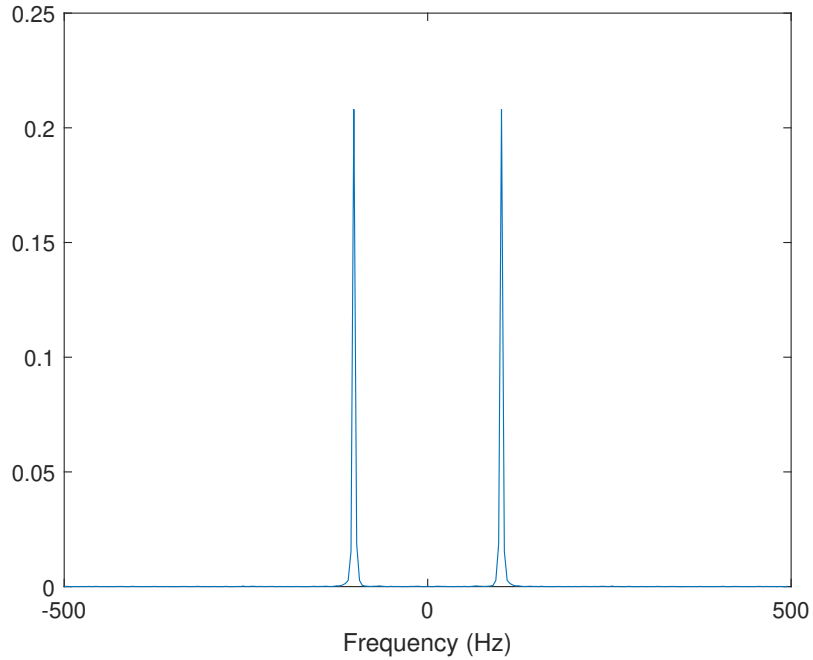
DFT vs. Spectrogram



Periodic phenomenon ? (sinusoid ?)

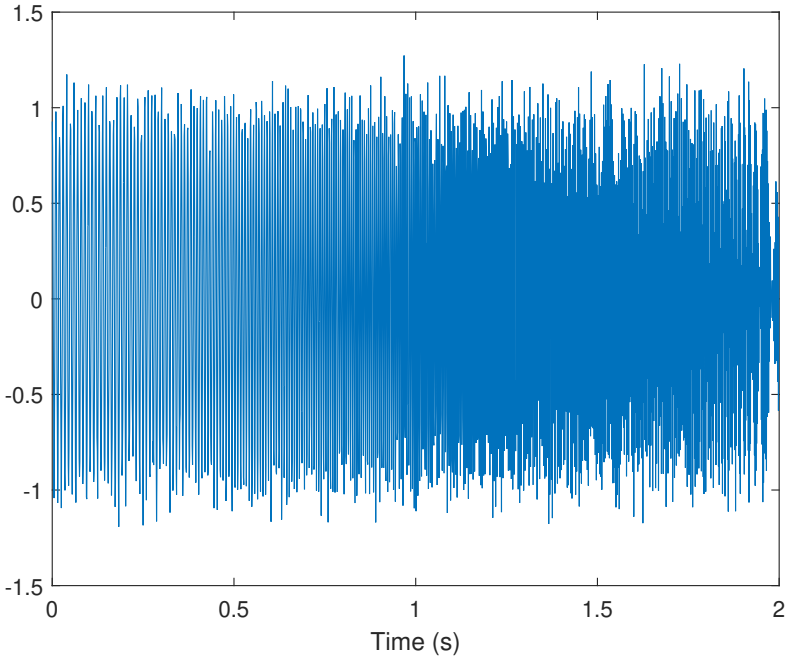
DFT vs. Spectrogram

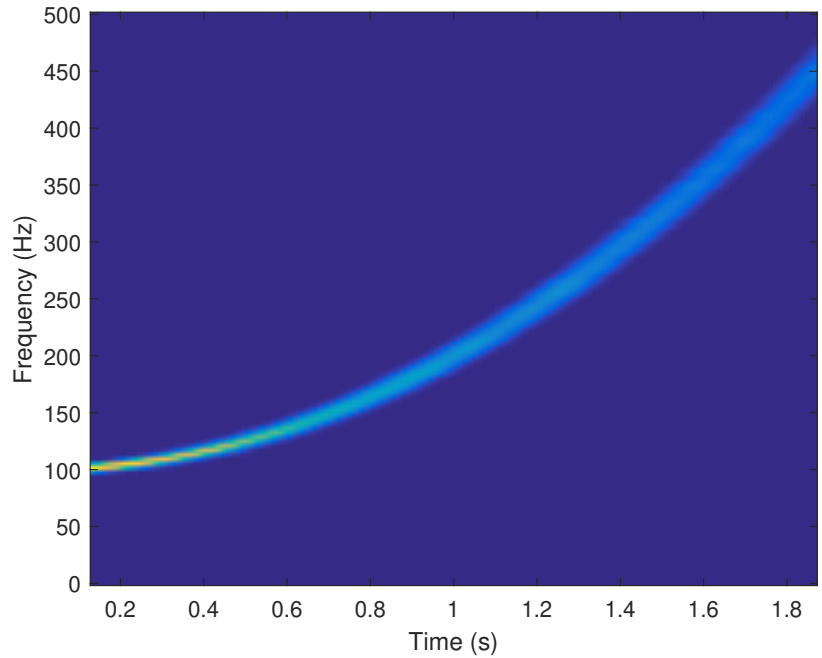




DFT suggests a sinusoidal phenomenon around frequency 100 Hz

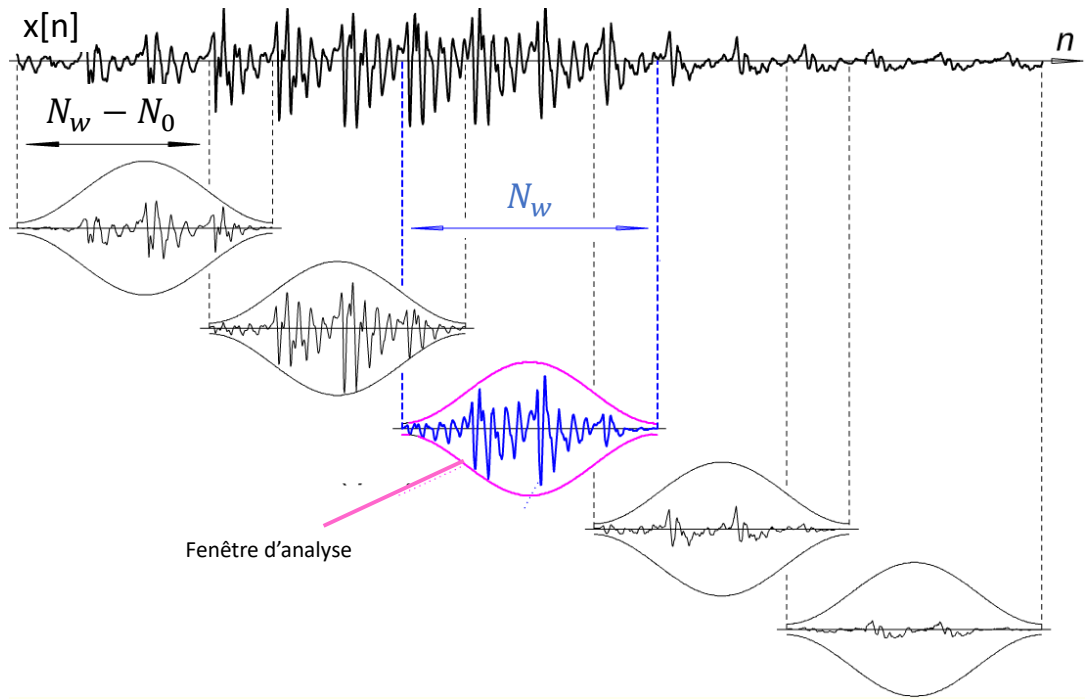
DFT vs. Spectrogram





In fact, chirp signal between 100 and 500 Hz !!

Hyperparameters for spectrogram



- N_w : **window length** (in samples)
  Often taken as a power of 2 (for FFT) and linked to the desired frequency resolution.
- N_o : **overlap** between two successive frames (in samples)
  Often taken as 50% or 75% of the window length and characterizes the time resolution (optimal when $N_o = N_w - 1$)
- w : **analysis window** (Hann, Hamming, Blackman...)
  Traditionally, in order to limit side effects, the signal frame is multiplied by an analysis window

Importance of the window length

- With a window of N_w samples, the time and frequency resolutions are

$$\Delta t \approx N_w T_s \qquad \Delta f \approx \frac{F_s}{N_w}$$

- Hence

$$\Delta t \cdot \Delta f \approx 1$$

- Temporal and frequency resolution **cannot both be improved**
  - **Long window**: small Δf, large Δt
    Two close frequencies are separated, but a brief event is spread over the whole window
  - **Short window** $\rightarrow$ small Δt, large Δf
    A brief event is precisely located, but two close frequencies merge into a single peak

Periodogram

- In reality, spectral estimation is a much more complicated subject than it seems... which is also linked to the statistical assumptions made on the signal
- When we compute the spectrum of a random signal, we assume that the signal is wide-sense stationary, and we estimate the so-called **Power Spectral Density (PSD)** $S_x[k]$ of the random process
- A natural spectral estimator of the PSD is called the **periodogram**

$$\hat{S}_x[k] = \frac{|X[k]|^2}{N}$$

- **Asymptotically unbiased** : $\mathbb{E}\left[\hat{S}_x[k]\right] \to S_x[k]$
- **Not consistent** : $\text{var}\left(\hat{S}_x[k]\right) \to S_x[k]^2$, which does not decrease with N ! Increasing N only adds more frequency bins, each just as noisy

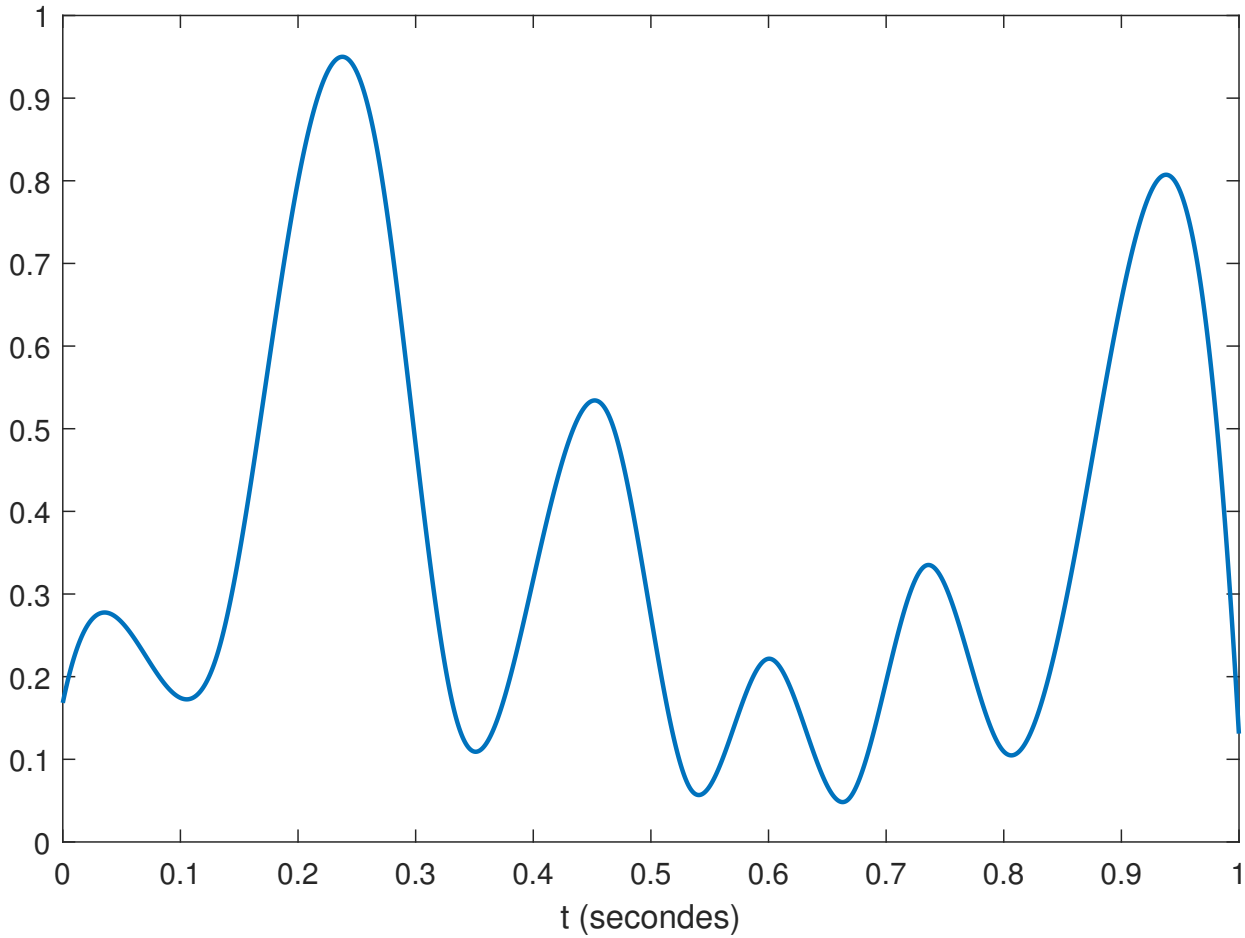
Variants for spectral estimation

In order to improve the spectral analysis, it is possible to rely on averaging techniques:

- **Bartlett** : average the periodograms of K non-overlapping segments (variance divided by K, resolution K times coarser)
- **Welch** : same with overlapping windowed segments (typically 50% overlap)
- **Multitaper** : average periodograms computed with several orthogonal analysis windows

Careful ! Averaging only makes sense for wide-sense stationary signals!

Local symbolic features



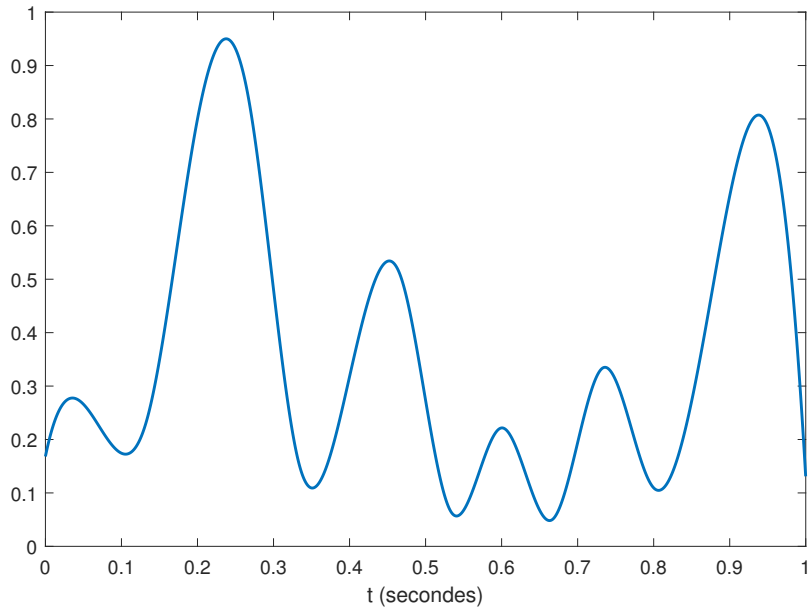
How would you describe such a signal?

small up, small down, big up, big down, up, down...

Local symbolic features

- By describing the whole time series as a succession of local behaviors, it is possible to fully encode the time series
- This encoding process is referred to as **symbolization**: it takes a time series as input and outputs a sequence of discrete symbols
- Each symbol represents a local behavior (amplitude, slope...) and statistical features can be extracted from that (number of occurrence of each symbol, number of transitions, etc...)
- These local symbolic features are the basis of several state-of-the-art time series classification techniques such as **BOSS** and variants [Schäfer et al., 2015]

Quantization/symbolization

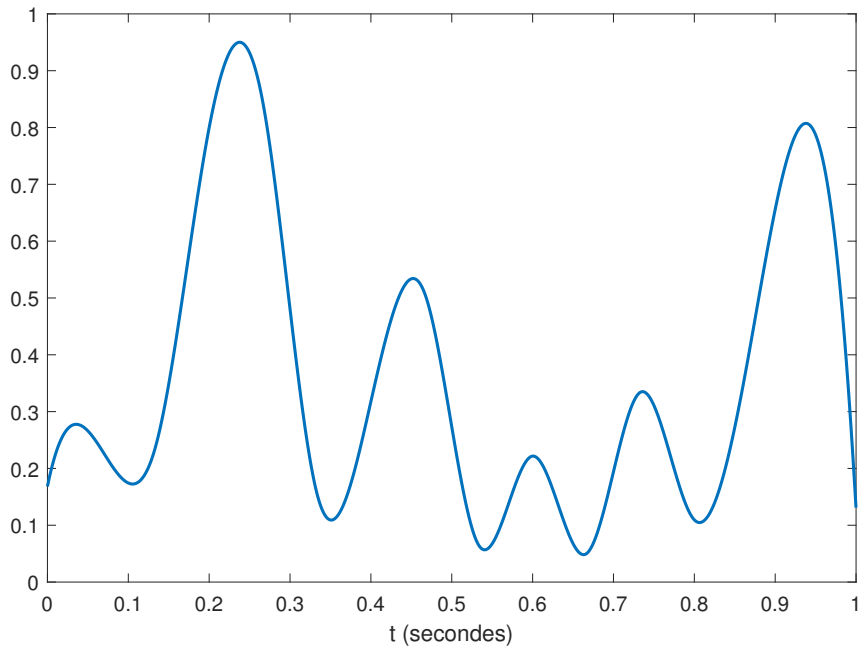


CCCFJHCBEEBBBACCBEHF

Quantization/symbolization

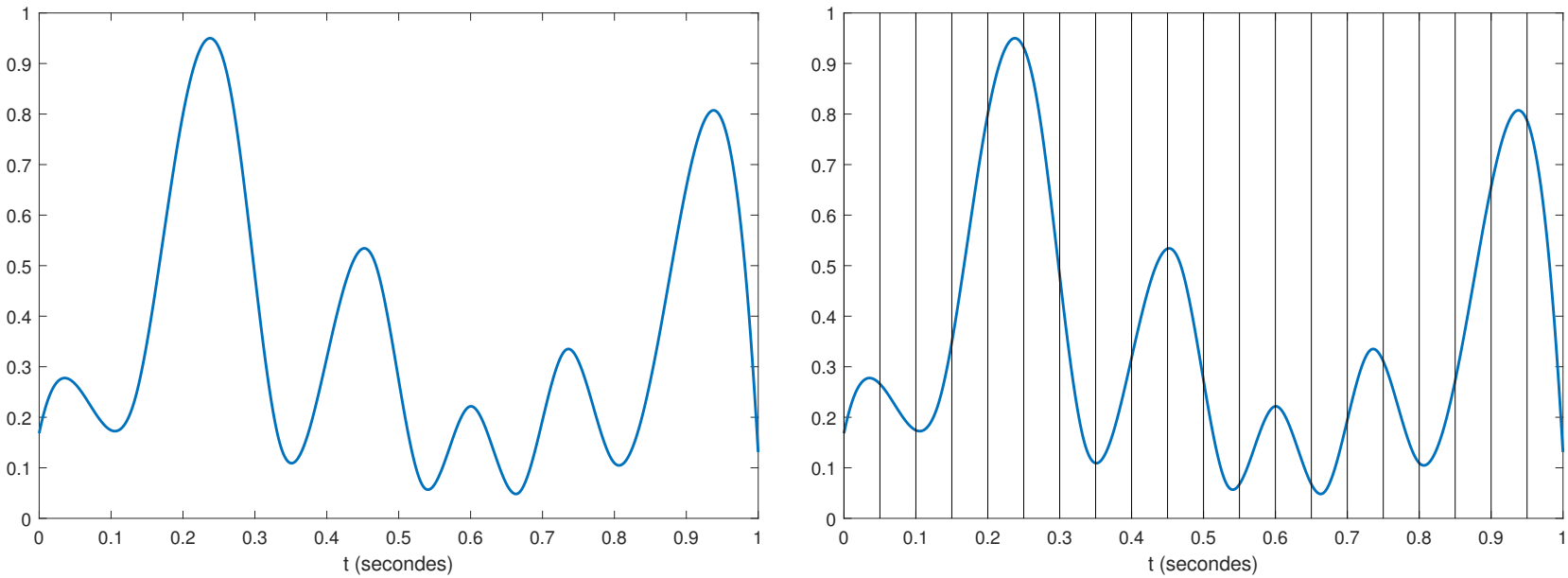
Given a real-valued signal x of length N, transform it into a discrete-valued time series y of smaller length.

Example



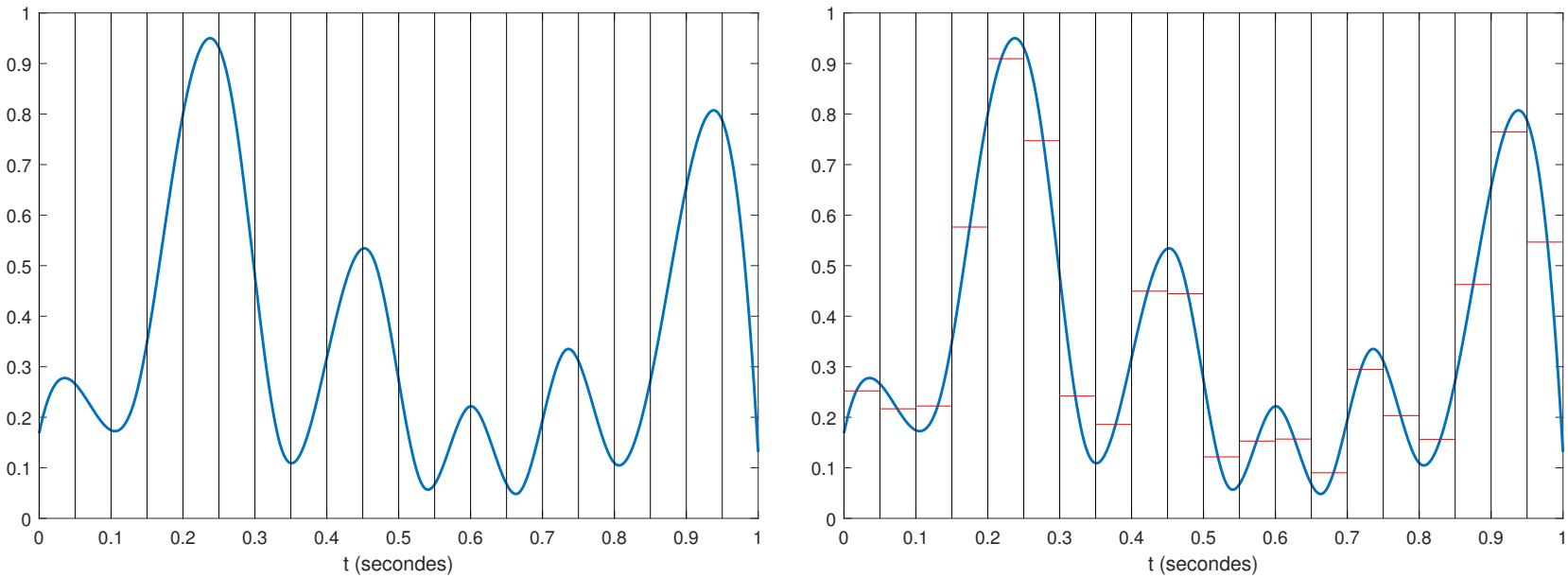
Original signal, sampled at $F_s = 10000$ Hz

Example



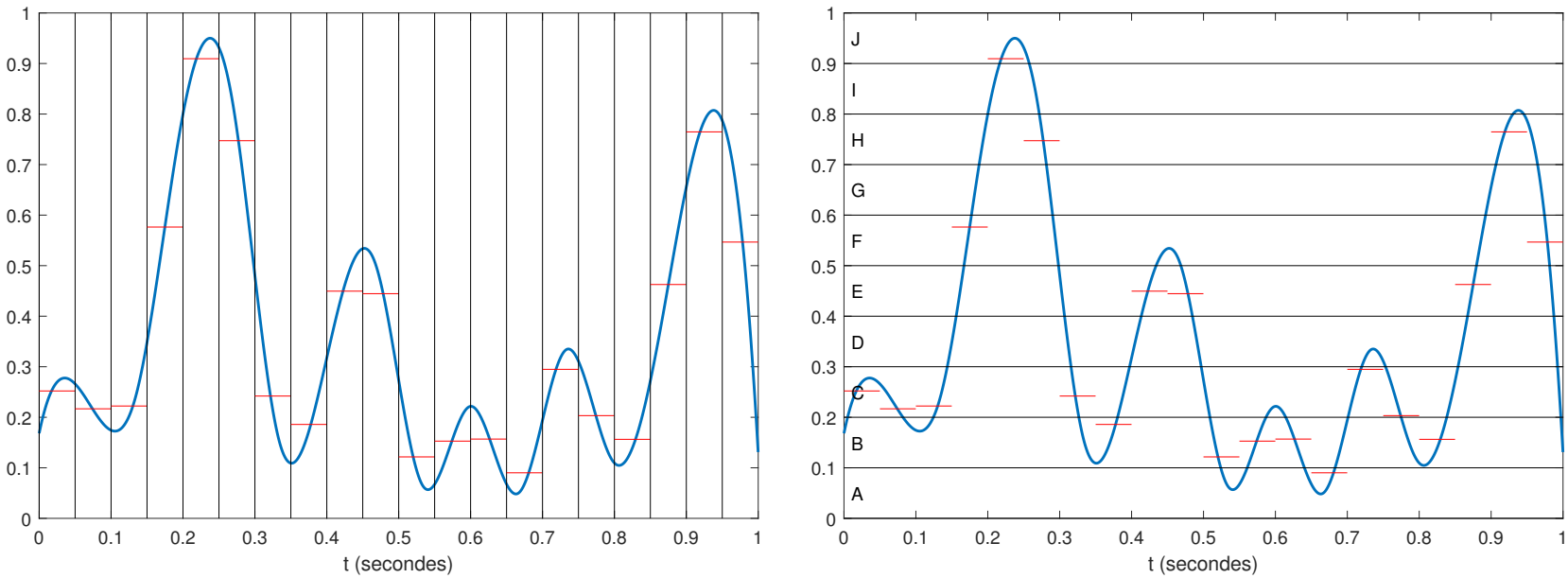
Division into non-overlapping frames of 500 samples

Example



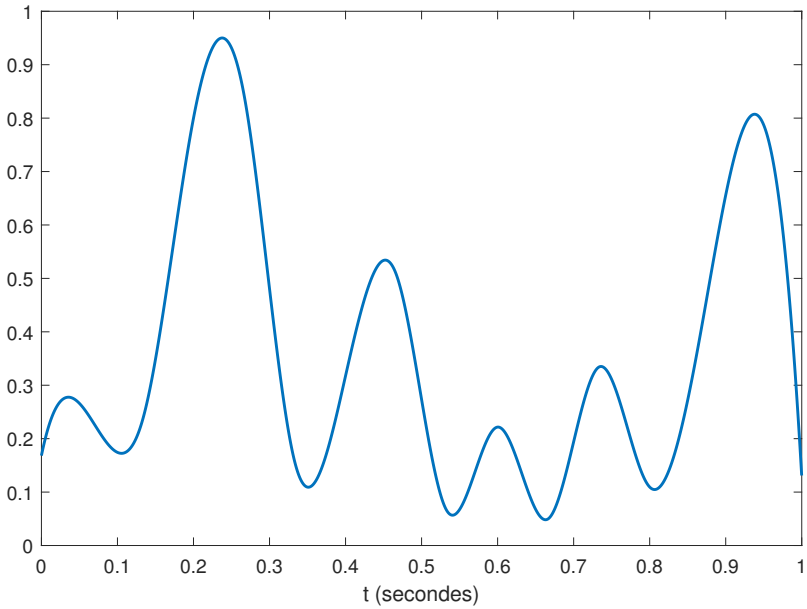
Average value on each segment

Example



Quantization of the amplitudes

Example



CCCFJHCBEEBBBACCBEHF

Symbolization of the time series
640 kbits vs. less than 0.1 kbit !

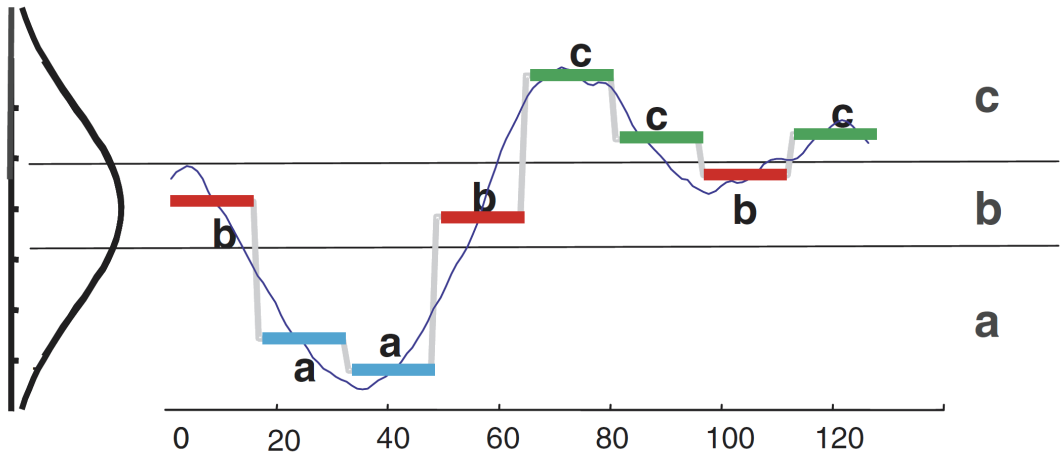
The SAX representation

- **SAX**, for Symbolic Aggregate approXimation [Lin et al., 2007], has been introduced as a strategy for computing such symbolization
- Input parameters
  - w : desired size of the output representation
  - A : number of symbols in the alphabet
- Each time series is encoded on $w \times \log_2(A)$ bits
- Authors have also developed a distance, called **MINDIST**, such that, given two time series $\mathbf{x}$ and $\mathbf{y}$:

$$\text{MINDIST}\left(\text{SAX}(\mathbf{x}), \text{SAX}(\mathbf{y})\right) \leq d_{EUC}(\mathbf{x}, \mathbf{y})$$

which preserves some classification properties on the symbolic space

The SAX representation



1. **Normalize** the time series (zero mean, unit variance)
2. **Divide** the time series into w segments and compute the mean
3. **Assign** one symbol per segment. The A quantization intervals are determined so that all symbols have the same probability according to a $\mathcal{N}(0,1)$ distribution.

How to construct features

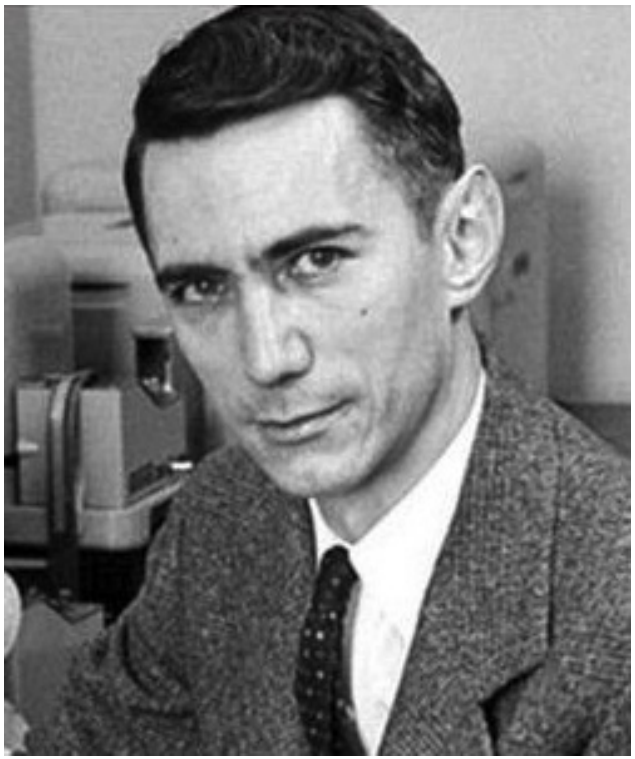
- Statistical features can be extracted from the discrete sequence of symbols (i.e. histograms, transition counts, histograms of n-grams...)
- The symbolization can be applied on sliding windows so as to form super-symbols that can be aggregated
- **Multiscale approaches** with different window sizes can also be used, and the histograms merged so as to provide a better representation
- Dozens of different symbolization procedures exist in the literature (encoding the slopes, the local Fourier coefficients, the min/max values etc...) : see mini-projects

Quantization/symbolization

Quantization/symbolization can be used for several other tasks

- It can reduce time complexity and storage size
- It allows **full enumeration** of some phenomena of interest (patterns, anomalies...)
- It allows us to use for time series several techniques from **NLP or bioinformatics**: pattern recognition, anomaly detection, etc... for discrete sequences

Information theory



- Shannon's information theory has emerged in the late 1940s as a **unified framework for representing digital information**
- One of the main breakthroughs that enabled the design of digital communications
- Data can be represented in **binary form**, can be compressed, sent with low error probability, etc : data compression, data encryption
- Each message contains a certain **quantity of information**, that is linked to its **probability of occurrence**

Entropy

- Given a random variable X in a finite alphabet $\mathcal{X}$, the **entropy** is defined as

$$H(X) = -\sum_{x \in \mathcal{X}} p_X(x) \log_2 \left(p_X(x) \right)$$

- It represents the average **quantity of information** contained in the variable
- **Maximum entropy** is achieved when X follows a **uniform distribution** : maximum uncertainty on sample values. In this case

$$H(X) = \log_2 \left(|\mathcal{X}| \right)$$

Entropy of two variables

Given two random variables X and Y, we can define:

- The **joint entropy** $H(X, Y)$: average information provided by the couple (X, Y)

$$H(X, Y) = -\sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p_{XY}(x, y) \log_2 \left(p_{XY}(x, y) \right)$$

- The **conditional entropy** $H(X|Y)$: uncertainty on X given Y

$$H(X|Y) = -\sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p_{XY}(x, y) \log_2 \left(p_{X|Y}(x|y) \right)$$

- The **mutual information** $I(X; Y)$: quantity of information shared by X and Y

$$I(X; Y) = \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} p_{XY}(x, y) \log_2 \left(\frac{p_{XY}(x, y)}{p_X(x) p_Y(y)} \right)$$

How to apply this to time series?

- First, time series samples are not necessarily discrete : in most cases, we have $x[n] \in \mathbb{R}$
  - **Solution 1**: Use quantization or symbolization techniques to transform the time series into a discrete sequence
  - **Solution 2**: Use an adapted distance to redefine the notion $\mathbb{P}(X = x)$ as $\mathbb{P}(d(X, x) \leq \epsilon)$
- Second, when computing entropy, **time information is lost**

$$\mathbf{x} = (1.6, 0.2, 1.6, 0.2, 1.6, 0.2, 1.6, 0.2)$$

$$\mathbf{y} = (1.6, 1.6, 1.6, 1.6, 0.2, 0.2, 0.2, 0.2)$$

have the same entropy!

Extension of a source

- Given a random variable (often referred to as a **source** in information theory) X on a discrete alphabet $\mathcal{X}$, we can define the **extension of degree** m of the source, denoted $X^{[m]}$, the source sending groups of m consecutive symbols $x_{i_1} x_{i_2} \dots x_{i_m}$
- Example : if X is a source on alphabet $\{A, B\}$, $X^{[2]}$ corresponds to the source sending symbols AA, AB, BA and BB.
- The source $X^{[m]}$ has an alphabet of size $|\mathcal{X}|^m$
- Its probability distribution can be estimated by computing the probability of occurrence of each subsequence of length m in the time series

Approximate entropy

The **Approximate entropy** [Pincus., 1991] of order m can be computed as follows:

1. Estimate the probability distribution for $X^{[m]}$ and $X^{[m+1]}$.
   - If the time series values are discrete this can be done by simple enumeration
   - If the time series values are real, use an alternate definition for the event $(X[n], X[n+1], \ldots, X[n+m-1]) = (x[n], x[n+1], \ldots, x[n+m-1])$

$$\max_{0 \leq i \leq m-1} |X[n+i] - x[n+i]| \leq \epsilon$$

   where ϵ is an additional parameter
2. Compute $H\left(X^{[m]}\right)$ and $H\left(X^{[m+1]}\right)$
3. The **Approximate Entropy (ApEn)** is computed as

$$\text{ApEn} = H\left(X^{[m+1]}\right) - H\left(X^{[m]}\right)$$

Intuition behind ApEn

Two limit cases

- When the samples are independent and identically distributed, it is easy to prove that $H\left(X^{[m]}\right) = mH(X)$ and the Approximate Entropy is maximal:

$$\text{ApEn} = H(X)$$

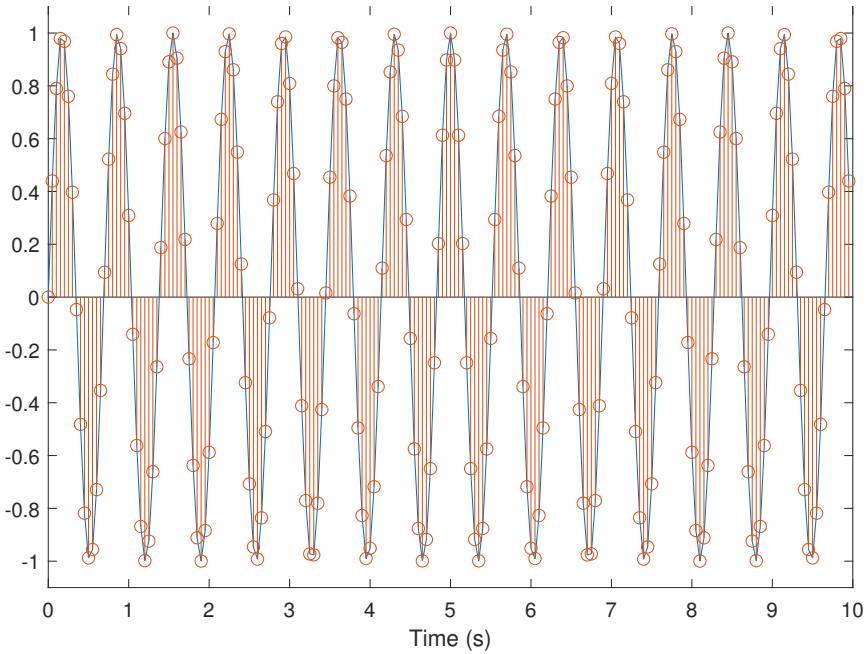
  It corresponds to the **maximum chaos** case: no temporal structure can be found in the time series

- When all samples are equal, the entropy is equal to zero and

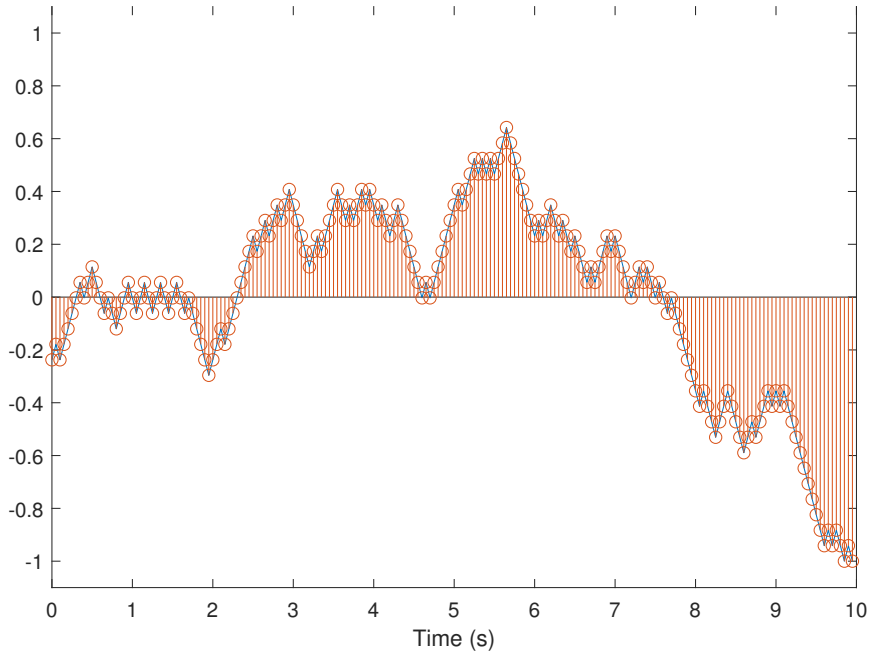
$$\text{ApEn} = 0$$

  On the contrary, there is no randomness in the data: everything is **deterministic**

Examples



Sinusoid
ApEn=0.2185



Random Walk
ApEn=0.4633

In general, ApEn is computed with $m = 2$ (to keep computation time limited)

Improvements

- Several improvements of the Approximate Entropy exist, such as the **Sample Entropy** (see one of the proposed mini-projects)
- In particular, ApEn estimates the probability of occurrence of a sequence but counts each sequence as matching itself, which can **introduce a bias**
- Theoretical guarantees are difficult to achieve when dealing with real-valued time series, and are crucially dependent on the number of samples

Global vs. local features

- Most of the features seen until now focus on **global** properties of the time series. Those are computed
  - **Directly from the signal values** : statistical features, spectral features
  - **Through a fixed vocabulary of local behaviors** : symbolic features, information theory features
- Another approach is to describe a time series by **the local shapes it contains**

Which local shapes are present, how strongly, how often, and where ?

- Complex problem that depends on
  - The **scale** of the shapes
  - The **features** extracted from the shape detection
  - How the shapes are **obtained**

How to detect local shapes

Let $\mathbf{w} \in \mathbb{R}^\ell$ be a filter and $\mathbf{x}$ a time series

$$z_i = \sum_{j=0}^{\ell-1} x[i+j]\, w_j + b$$

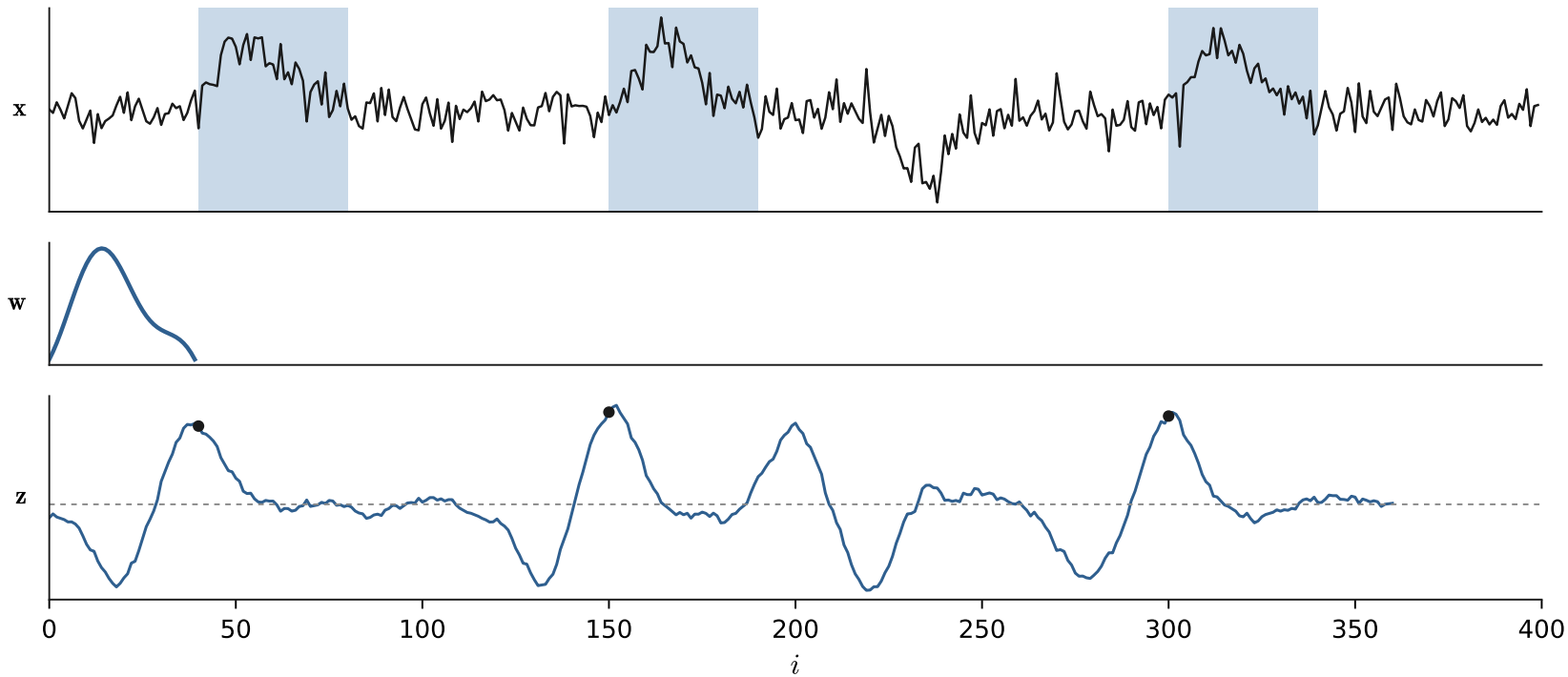
- z_i measures the resemblance between $\mathbf{w}$ and the subsequence starting at i
- The vector $\mathbf{z} = (z_1, \ldots, z_m)$ is called the **activation map**
- It can be computed as the **convolution** of $\mathbf{x}$ with the time-reversed filter $\mathbf{w}^{\leftarrow}$ (see Lecture 1), hence in $\mathcal{O}(N \log N)$ by FFT, whatever ℓ

Exactly the sliding operation of Lecture 1 (distance profile) :

$$d^2_{EUC}(\mathbf{w}, x[i : i+\ell-1]) = \underbrace{\|\mathbf{w}\|^2 + 2b}_{\text{constant}} + \|x[i : i+\ell-1]\|^2 - 2z_i$$

At constant local energy, maximizing z_i = minimizing the distance

Example



Three occurrences of the pattern give three peaks in the activation map

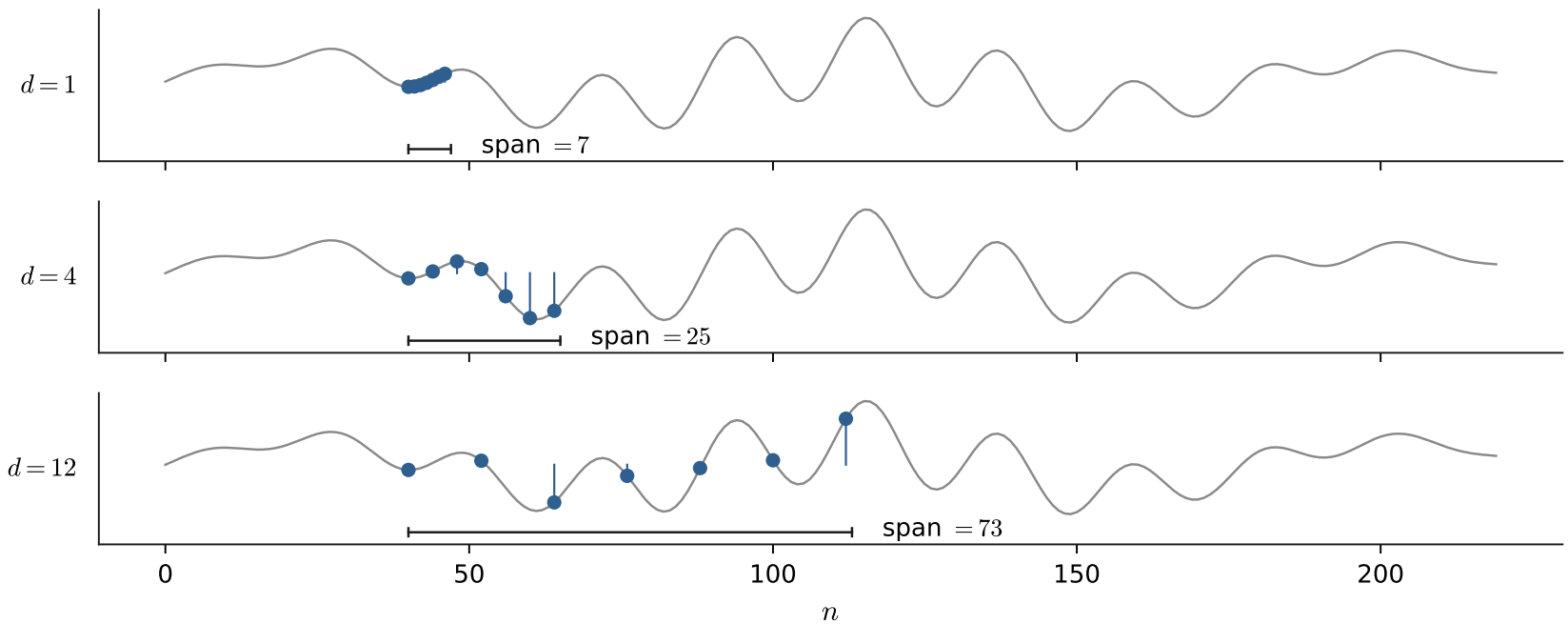
Notion of scale

- How to choose ℓ ? Real patterns may occur at **different scales**, unknown in advance
- Increasing ℓ means more parameters and a higher computational cost
- **Dilation** : insert gaps of $d-1$ samples between consecutive taps

$$z_i = \sum_{j=0}^{\ell-1} x\left[i + d \cdot j\right] w_j + b$$

- Spans $d(\ell-1)+1$ samples, but still only ℓ multiplications : detects the same shape **subsampled**, i.e. at a coarser time scale

Example

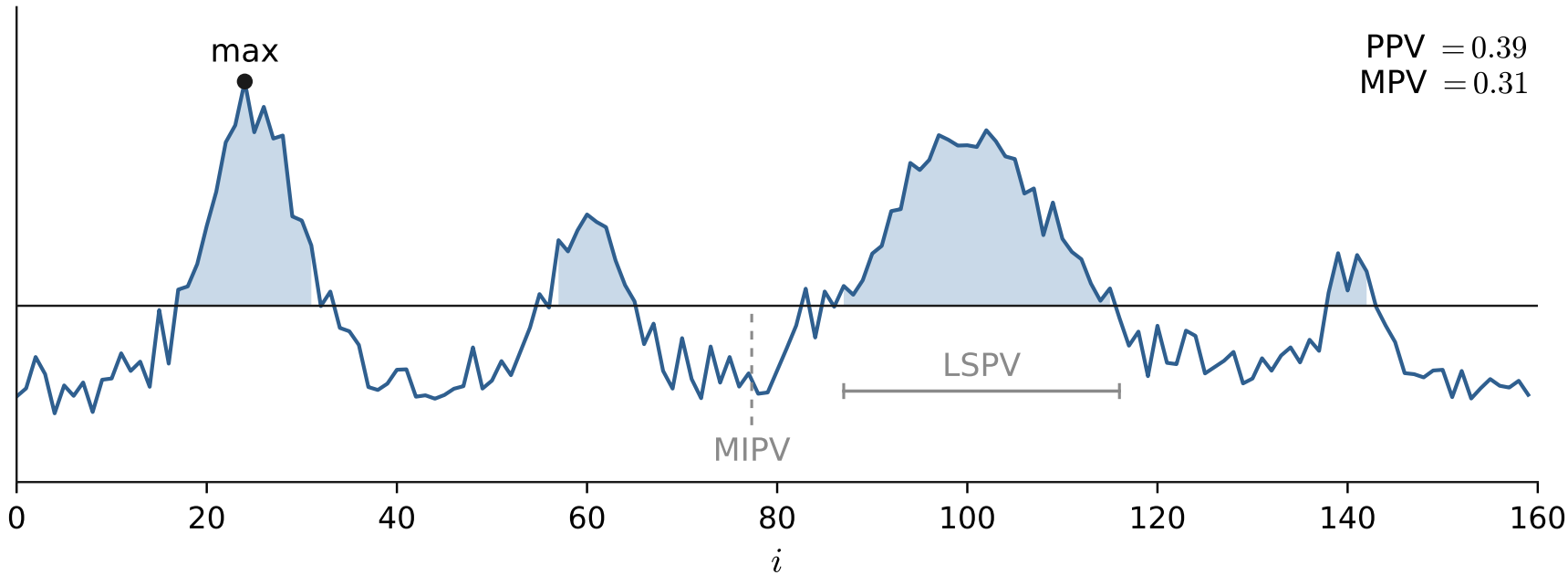


The same filter of length $\ell = 7$ at three time scales: the number of parameters does not change

Extracting features

- $\mathbf{z}$ has the length of the time series, but a feature must be a scalar
- The idea is therefore to extract features from the activation map, that reflect different properties
  - **max** : $\max_i z_i$, the strength of the best match
  - **PPV** (proportion of positive values) : $\frac{1}{m}\sum_i \mathbf{1}\{z_i > 0\}$, the proportion of positions where the pattern is detected
  - **MPV** (mean of positive values) : $\sum_i z_i \mathbf{1}\{z_i > 0\} / \sum_i \mathbf{1}\{z_i > 0\}$, the average strength of the detections
  - **MIPV** (mean of indices of positive values) : $\sum_i i\, \mathbf{1}\{z_i > 0\} / \sum_i \mathbf{1}\{z_i > 0\}$, the average position of the detections
  - **LSPV** (longest stretch of positive values) : $\max\{j - i : z_k > 0\ \forall k \in [i, j]\}$, the longest uninterrupted detection

Example



Several features can be computed from the activation map

PPV: proportion of positive values MPV: mean of positive values

MIPV: mean of indices of positive values LSPV: longest stretch of positive values

Properties of the activation maps

- Note that, depending on the features used, some are invariant with respect to translation (e.g. max) and some use positional information (e.g. MIPV)
- Instead of using an offset equal to 0, it is possible to use adaptive offsets b that depend on the distribution of values in the activation maps

$$\text{PPV}(\mathbf{z}) = \frac{1}{m}\sum_i \mathbf{1}\big\{\langle \mathbf{w}, x[i : i+\ell-1]\rangle > -b\big\} = 1 - \widehat{F}(-b)$$

where $\widehat{F}$ is the empirical CDF of the activation map

Invariance by construction

Instead of normalizing the data, it is possible to constrain the **filters**

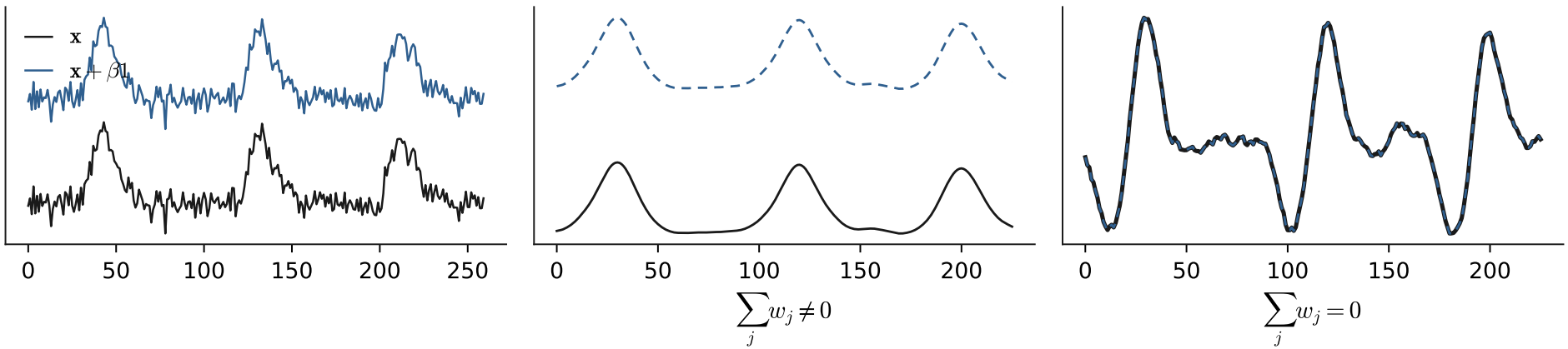
$$\sum_j w_j = 0$$

Then for any offset β :

$$\sum_j \big(x[i+dj] + \beta\big) w_j = \sum_j x[i+dj] w_j + \beta \underbrace{\sum_j w_j}_{=0} = \sum_j x[i+dj] w_j$$

- Every activation, hence every feature, is unchanged by an additive offset
- This is the convolutional counterpart of subtracting the mean in z-normalization (see Lecture 1)

Example

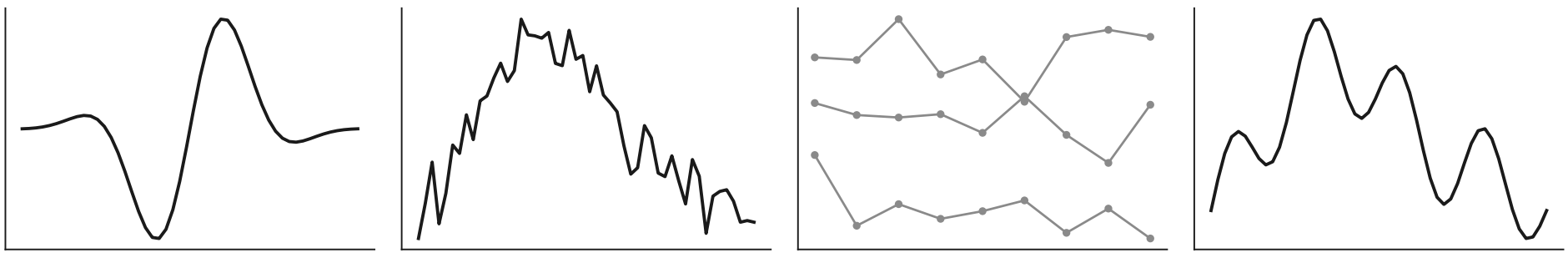


Left: two series differing by an offset. Right: with zero-sum weights, the two activation maps coincide exactly

Choice of the filters $\mathbf{w}$

- Four main solutions
  - **Manually picked** : given by domain knowledge (annotated data, wavelets, hand-designed filters for trends and peaks)
    Maximal interpretability, no learning but limited to what was anticipated
  - **Extracted from the data** : subsequences extracted from pattern extraction (it is the principle of the **shapelet** framework)
    Realistic filters but requires to run an extra unsupervised detection algorithm
  - **Random** : sampled from a distribution over filters, independently of the data
    No search and no learning but most are likely to be useless
  - **Learned** : parameters of a model optimized for a task
    Maximum performances but needs training data
- Interestingly, random filters have proven to be very useful and competitive

Example



Why do random filters work?

- This is an instance of the **random features** paradigm : a feature map $\Phi(\mathbf{x}) \in \mathbb{R}^D$ with D large and fixed randomly, followed by a **linear** model, approximates a rich nonlinear function class
- Instead of looking for the good filters, it is possible to use a very large number of random ones : with $D \sim 10^4$ features, some of them are likely to be informative
- The linear model then gives more weight to the informative ones : this is a supervised feature selection (see next part !)
- Two practical consequences :
  - $D \gg M$: it is necessary to use **ridge regression** or regularized techniques
  - Feature standardization is mandatory : max, PPV and MIPV live on different scales

Deep learning implementation

- To easily implement these ideas, one solution is to use **standard deep learning architectures** that are naturally adapted to compute convolutions
- Note that, while the computation of the activation maps and of the features is unsupervised, **most of the features are likely to be redundant or irrelevant**
- The strong performance achieved with these features comes from the combination of this unsupervised phase and a linear classifier, **which is supervised** (and requires annotated data)
- These convolutional features are therefore reserved for supervised tasks and are difficult to use, for example, for clustering.

Warning !!

Warning ! This will not be a proper up-to-date "Deep learning for time series" class !

Only the basic ideas will be provided

From fixed filters to learned filters

- Everything so far could be implemented **without any deep learning**
- It is also possible to learn them by using deep learning: in this case the filters will become **parameters**
- The idea is to encode the time series with a parametric function

$$\Phi_\theta : \mathcal{X} \to \mathbb{R}^D$$

  where θ are parameters adjusted by optimization
- The feature extraction becomes **supervised** and therefore needs annotated data

The basic ingredients for deep learning

Φ_θ is built as a (possibly very complex) combination of elementary operations:

- **Local shape detection** : convolutional layers (see previous section), $z_i = \sum_j x[i + d \cdot j]\, w_j + b$
- **Dilation/adaptation to scale** : dilation d inside the convolution, which stretches the filter without adding parameters
- **Pooling/dimensionality reduction** : pooling layers, turning a map into a shorter map (max, average) or into a scalar (global pooling), which also makes the output size independent of the input length
- **Nonlinear activation** : activation functions (ReLU, tanh)
- **Memory storage** : recurrent cells (RNN, LSTM, GRU) maintain a memory $\mathbf{h}_t$ updated sample by sample, $\mathbf{h}_t = f(\mathbf{h}_{t-1}, x_t)$
- **Attention mechanism** : attention layers, where each sample looks at every other one and decides how much to borrow from it

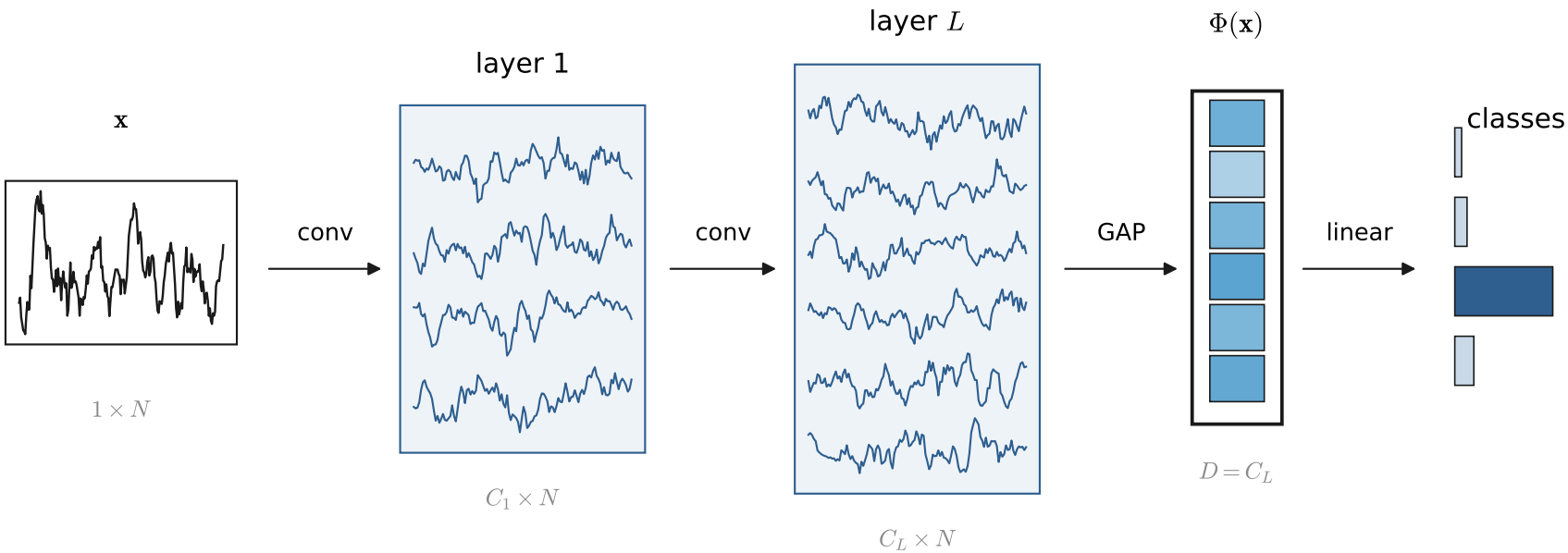
Where are the features in a network?

- A network is a stack of transformations: the representation is simply the output of an **intermediate layer**
- In practice, most papers use the layer just before the classifier, which is almost always a global average pooling(GAP)

$$\text{GAP}(\mathbf{z})_c = \frac{1}{m}\sum_{i=1}^{m} z_c[i] \quad \text{for each channel } c$$

- **Two consequences**:
  - The output has a **fixed size** whatever the length of the input
  - Averaging over time gives **invariance to translation**

Example



Example on a convolutional neural network

Three regimes of supervision

The choice of the loss and of the task defines what the features encode

- **Supervised** : the network is trained to predict a label, and the representation is a by-product
  - The features are optimal for that task, and may transfer poorly to another one
  - Requires annotations, which are the scarce resource for time series
- **Reconstruction-based** : no labels, the representation is required to be sufficient to rebuild the input
  - Only the information needed for the reconstruction is kept
- **Self-supervised** : no labels, a **pretext task** is designed
  - The task itself is of no interest : it is only used to force the encoder to represent a given type of information

The autoencoder principle

The principle is to simultaneously learn two nonlinear maps

- $\phi : \mathcal{X} \to \mathbb{R}^D$ that extracts the features (encoder)
- $\psi : \mathbb{R}^D \to \mathcal{X}$ that reconstructs the time series (decoder)

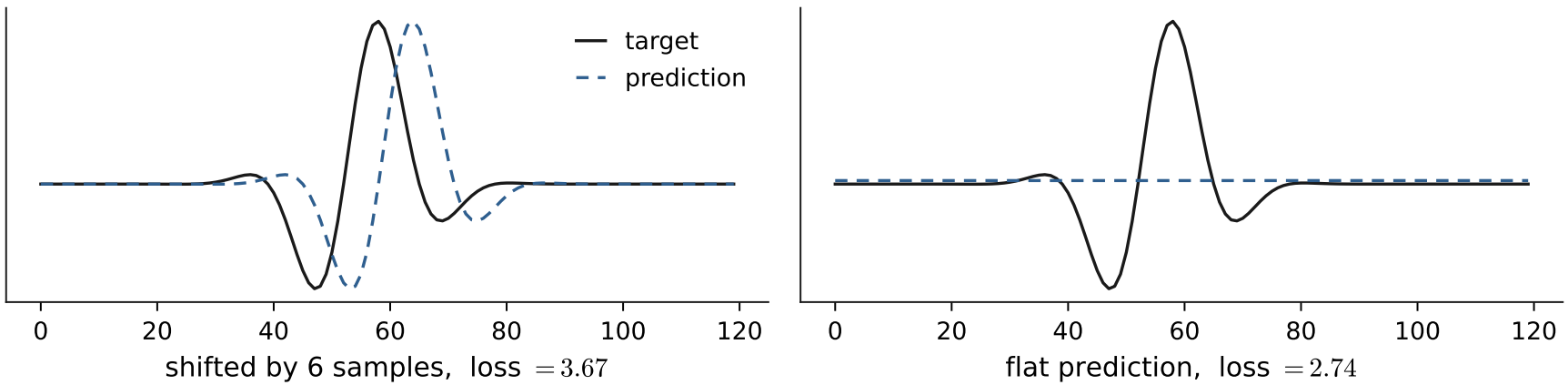
$$\phi^\star, \psi^\star = \text{argmin}_{\phi,\psi} \|\mathbf{X} - (\psi \circ \phi)(\mathbf{X})\|^2$$

- The time series is first cut into (possibly overlapping) frames of length N_w, stored in $\mathbf{X}$
- Two parameters only : the frame length N_w and the desired number of features D
- With $D \ll N_w$, the input cannot be copied, and only the information needed to rebuild it is retained

Importance of the loss

- Perfectly reconstructing a waveform is often complex and not always a good idea: the features will encode everything needed for reconstruction, including what we would like to discard (noise, exact amplitudes, exact positions)
- Choosing the right loss is important (eg : Euclidean cost penalizes a perfectly shaped reconstruction shifted by one sample as heavily as a completely wrong shape : see Lecture 1)
- Choosing a loss amounts to choosing which deformations are considered as irrelevant (see Lecture 1)

Example

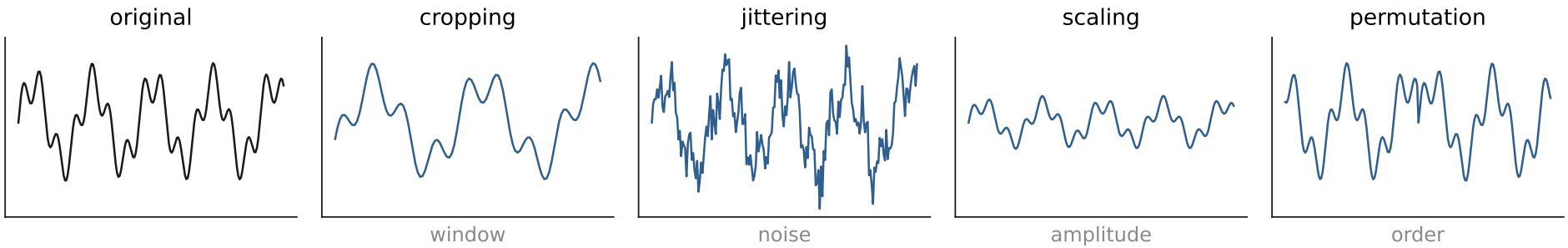


Importance of the loss: Euclidean distance is the reference loss... but it is not always the most adequate for time series!

Self-supervised techniques

- Rather than reconstructing, ths idea is to ask the encoder to solve a proxy task we **design**.
- The most popular approach is linked to **contrastive learning**
  - We feed to the network pairs of time series: similar time series should get similar representations, dissimilar times series should get dissimilar representations
  - Often relies on data augmentation techniques where we deliberately generate deformed versions of the original time series
  - Construction of **contrastive losses** that will make sure that those desired behaviors are favored
- An alternative pretext task is **masked modelling** : hide part of the input and reconstruct only the hidden part, which forces the encoder to use the context instead of copying the input

Augmentations and invariances



- Choosing the augmentation set specifies the desired invariance (still linked to Lecture 1!)
- This is the same modelling decision as choosing an invariance for a distance
- **Careful !** An augmentation that destroys task-relevant information degrades the features, without any visible symptom

Other features

Of course, the number of features can be unlimited !

- **Topological data analysis:** notion of invariants and extraction of topological data features
- **Wavelet analysis :** wavelet coefficient (time-scale analysis)
- **Models :** model parameters can be used as features (see Lecture 3)

Contents

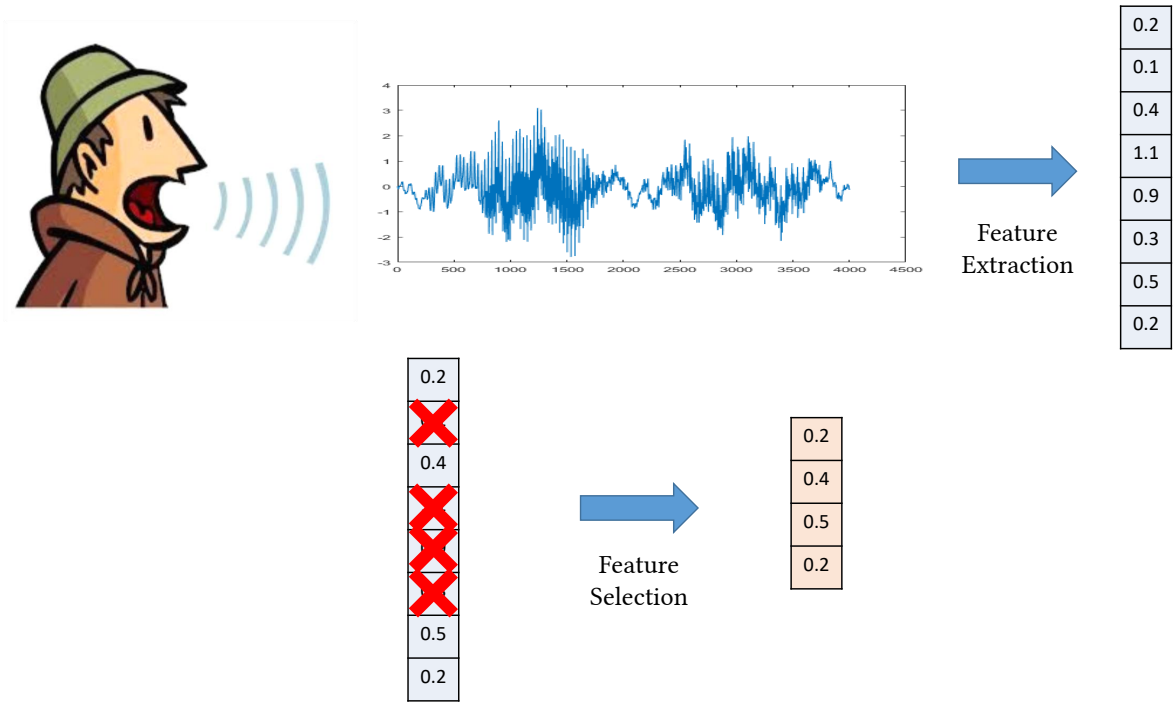
1. Problem statement

2. Feature Extraction

3. Feature Selection
3.1 Unsupervised setting
3.2 Supervised setting

4. Conclusion

Problem 2: Feature Selection



- **Given a dataset of M time series x represented each with D features, select the $K < D$ features that are relevant for a given task**
- Can be performed in a **supervised** (labeled data) or **unsupervised** setting

What is a good feature?

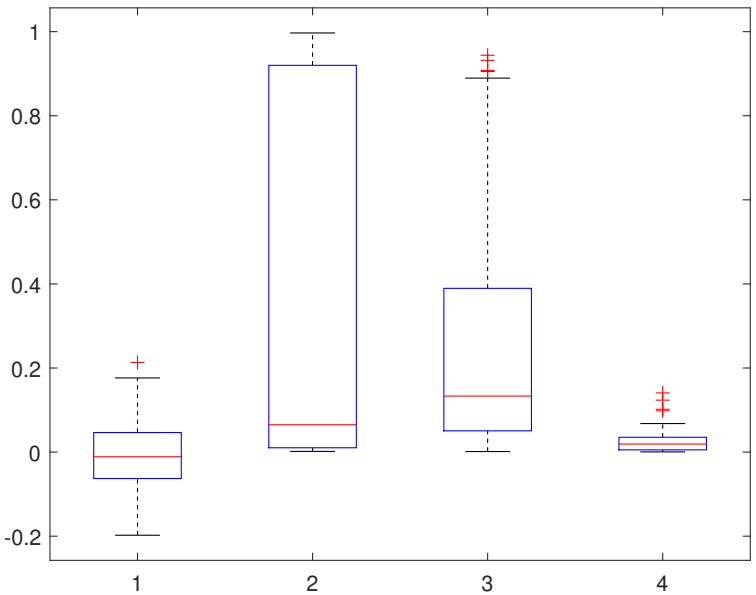
- Are they **robust**, **well calculated** and **relevant**?
- Do they reveal the differences **actually present** in the database?
- Do they have **good generalization properties**?
- **Notations**
  - M : number of observations
  - D : number of features
  - $\mathbf{x}_1, \ldots, \mathbf{x}_M$: observations
  - Data matrix $\mathbf{X} \in \mathbb{R}^{D \times M}$
  - Annotations (if available) $\mathbf{y} \in \{-1, +1\}^M$

Unsupervised case

Empirical observations:

- **Reliability** : presence of outliers?
- **Utility of the features** : do they allow to see differences between the observations?
- **Visualization and clustering**: can we distinguish groups in the database?

Reliability



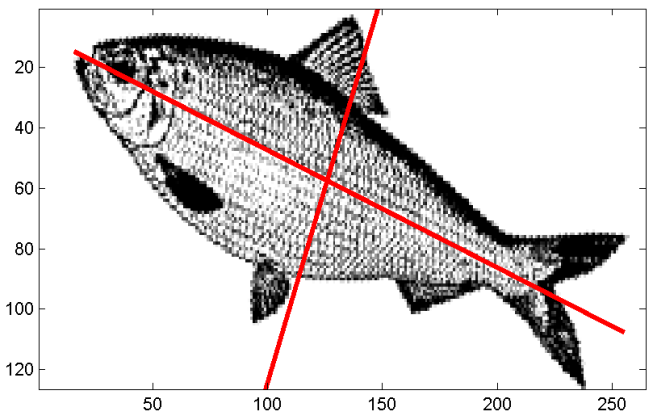
Boxplot: visualization of the quartiles

- Simple statistical **sanity checks**
  - Outlier detection by comparing the values to the standard deviation on all observations

$$\left| X_{d,m} - \mu_{X_{d,:}} \right| > 3 \times \text{std}\left(X_{d,:}\right)$$

  - Computation of 5% and 95% percentiles and comparison with minimum/maximum values
  - Simple variance test to detect features that are almost constant on the database
- The main idea is to check if extreme values **are due to the properties of the observations** or to any processing flaws (impulsive noise in the time series, bad feature computation, non-stationarities...): see Lecture 4 on Pre-processing

Utility of the features



Principal Component Analysis (PCA)

- **Principle**: detect the features that contribute the most to the global data variance
- Linear transform of the D initial features into D new features called **principal components**, which are uncorrelated

$$\tilde{f}_j = \sum_{d=1}^{D} U_{d,j} f_d$$

- Principal components $\tilde{f}_j$ are **ordered according to their contribution to the global data variance**: usually, we plot the data on the plane defined by the first two principal components

PCA for feature selection

Principal Component Analysis (PCA) to detect features that contribute the most to the data variance [Wold et al., 1987 ; Song et al., 2010]

1. Normalization of the data matrix so that each row (i.e. feature) is zero-mean and unit-variance

$$\tilde{X}_{d,:} = \frac{X_{d,:} - \mu_{X_{d,:}}}{\sigma_{X_{d,:}}}$$

2. **Singular Value Decomposition (SVD)** of $\tilde{\mathbf{X}}$:

$$\tilde{\mathbf{X}} = \underbrace{\mathbf{U}}_{D \times D} \underbrace{\mathbf{\Sigma}}_{D \times M} \underbrace{\mathbf{V}^\top}_{M \times M}$$

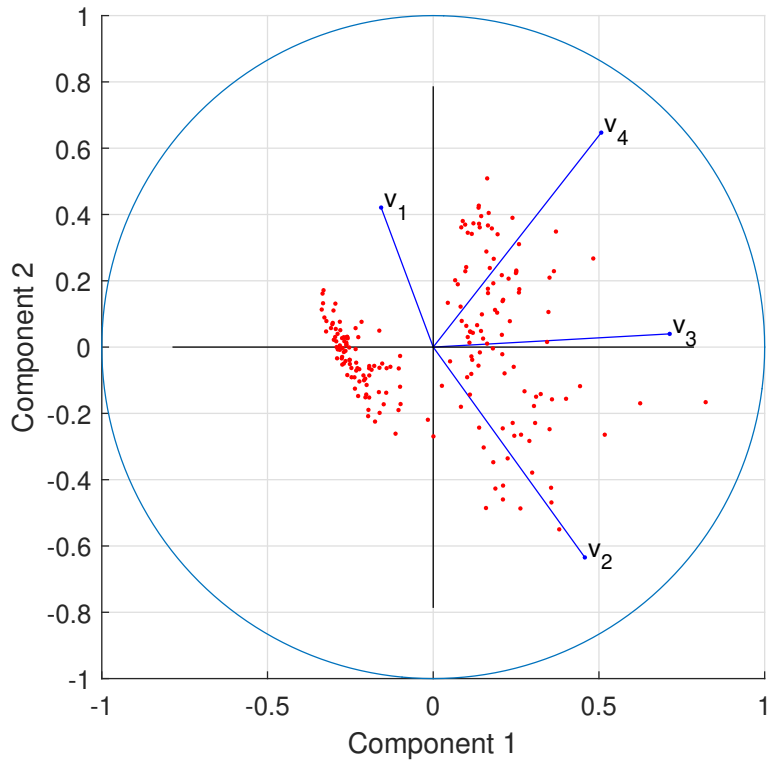
3. Plot of the contribution of each feature to the two first principal components on the **PCA correlation circle**

$$U_{d,j} : \text{ contribution of feature } d \text{ to the } j^{th} \text{ principal component}$$

4. Plot of the data points on the two first principal components. By denoting $\mathbf{S} = \mathbf{\Lambda V}^\top$

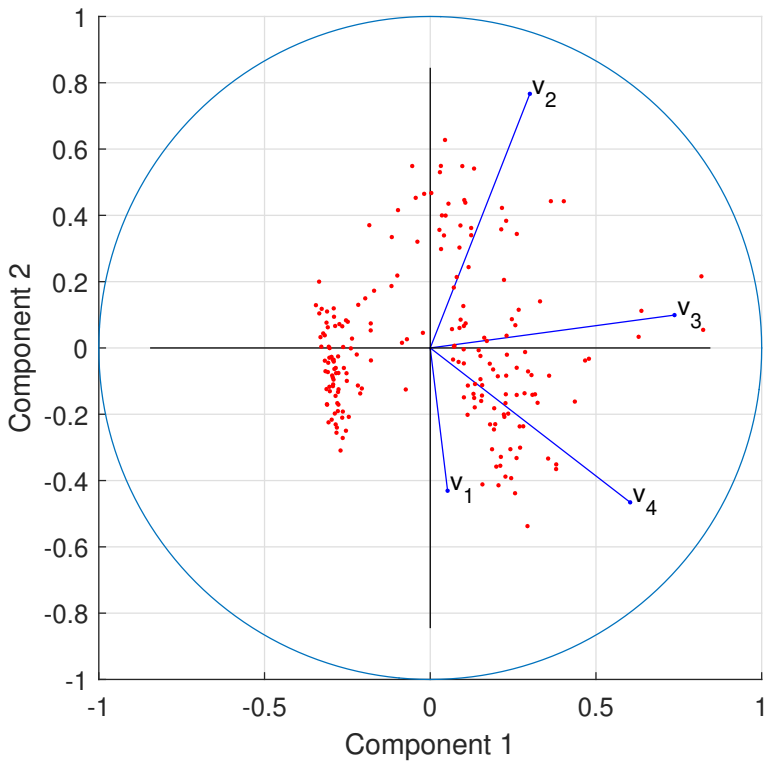
$$S_{j,m} : \text{ projection of observation } m \text{ on the } j^{th} \text{ principal component}$$

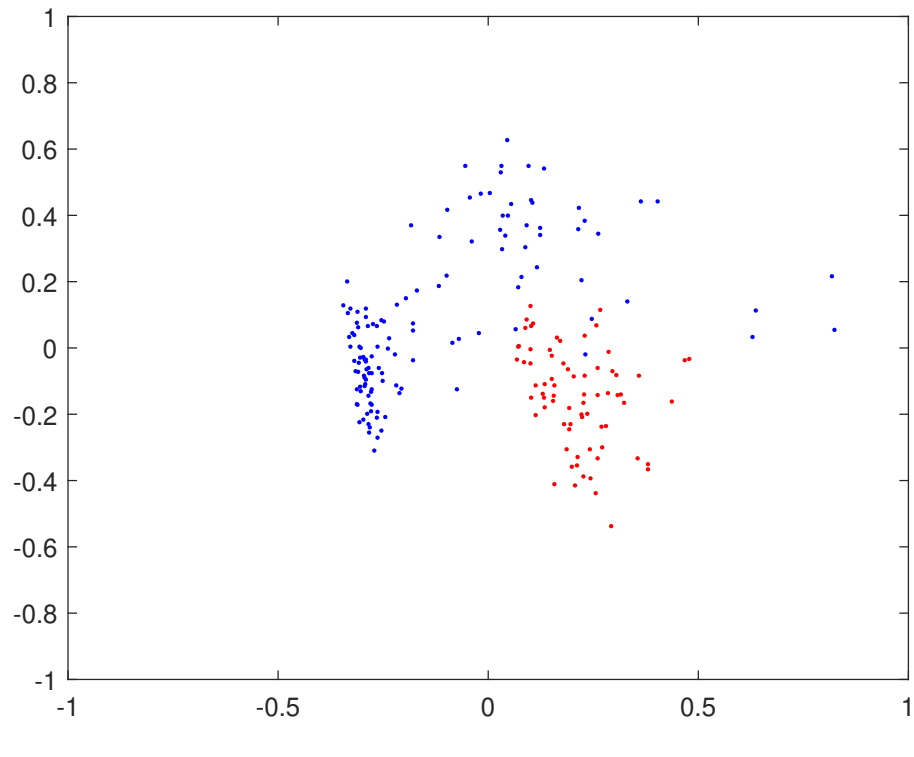
PCA visualization



- **Good feature** : close to the unit circle
- Allows us to see redundant features and orthogonal features

Clustering?





- If we know that several classes are supposed to exist in the dataset, it is possible use a **standard clustering algorithm** (k-Means for instance) to visualize the obtained clusters
- Distances of the data points to the centroids can be used to assess the relevance of the feature representation

Supervised setting

- We seek to find the features that are **in accordance with label information**
- Three main classes of methods [Li et al., 2017] :
  - **Filter methods :** We test the adequacy of the feature with the annotations, thanks to several criteria/scores (e.g. correlation).
  - **Wrapper methods :** We test the features on a supervised classification task, by trying several combinations. The best features are kept.
  - **Embedded methods :** Mixed approaches where we jointly infer the relevance of the features and classify the data (decision tree, sparse methods...)

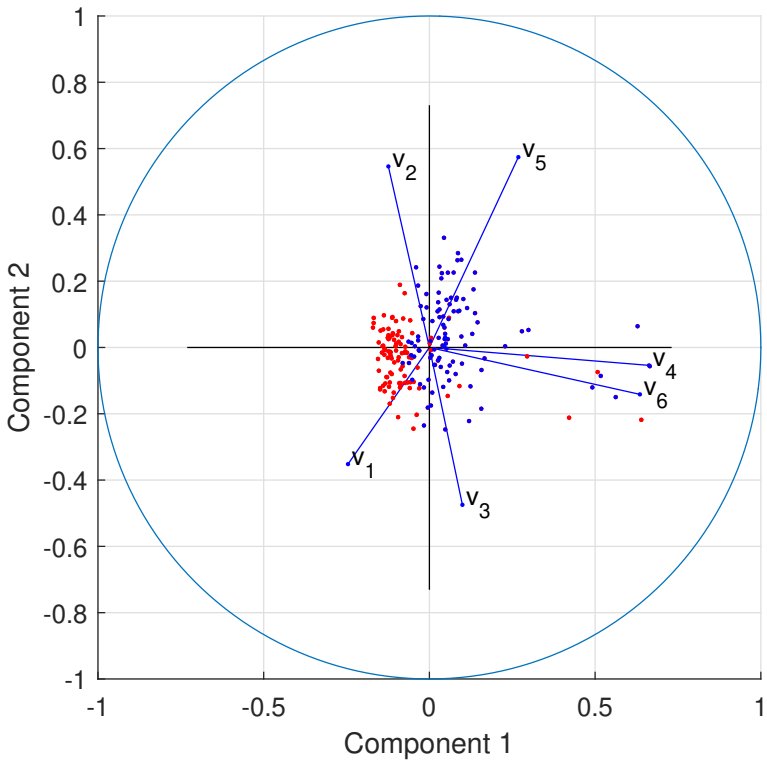
Filter methods

- We test (often individually) the relevance of each feature by studying its link with the annotations
- **Correlation score**. Computation of the Pearson correlation coefficient between the data vector $X_{d,:}$ of feature d and the annotation vector $\mathbf{y}$

$$\rho(d) = \frac{\text{cov}\left(X_{d,:}, \mathbf{y}\right)}{\sqrt{\text{var}\left(X_{d,:}\right)\text{var}\left(\mathbf{y}\right)}}$$

  - This coefficient is comprised between -1 and +1 and allows us to see the linear correlation between observations and annotations
  - We keep all features for which $|\rho(d)| > \lambda$: threshold-based selection of the $K < D$ best features
  - **Careful !** Since we select the feature one-by-one, **there might exist some redundancy** between highly correlated features !

Example



- Two classes of signals and $D = 6$ features
- According to PCA (unsupervised), features v_2, v_4, v_5, v_6 appear to be relevant
- By computing the correlation scores (supervised), we have

$$-0.02 \quad 0.08 \quad 0.32 \quad 0.45 \quad 0.82 \quad 0.23$$

Features v_4, v_5 are the most relevant for the classification task

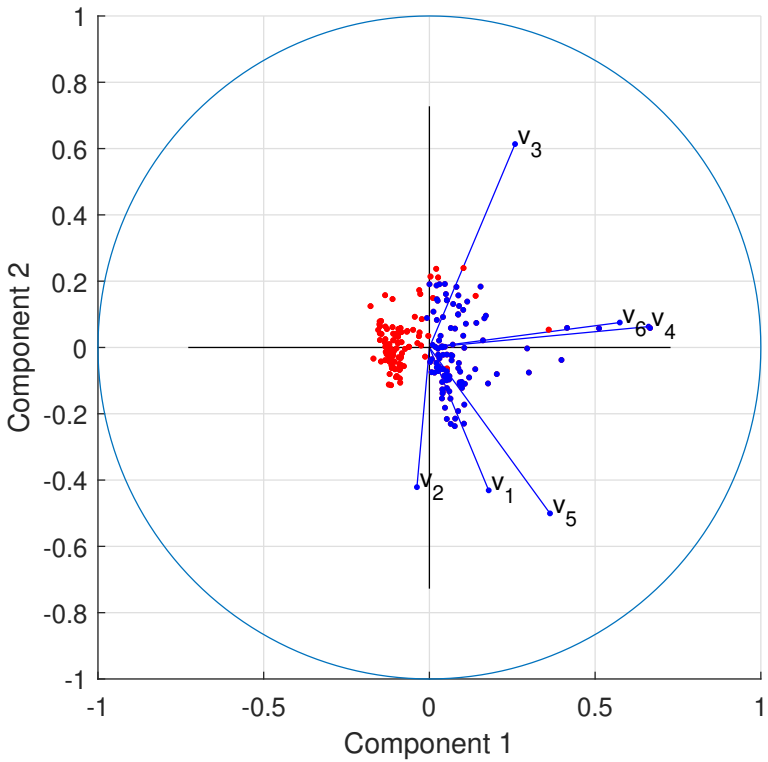
Filter methods

Fisher score. [Gu et al., 2011]

- Multiclass labels $\mathbf{y}$ with C classes
- n_c : number of data points in class c
- $\mu_{c,d}, \sigma_{c,d}$: mean/standard deviation of feature d for data point in class c
- μ_d : mean value of feature d on the whole database

$$\text{fisher_score}(d) = \frac{\sum_{c=1}^{C} n_c \left(\mu_{c,d} - \mu_d\right)^2}{\sum_{c=1}^{C} n_c \sigma_{c,d}^2}$$

Example



- Two classes of signals and $D = 6$ features
- According to PCA (unsupervised), features v_3, v_4, v_5, v_6 appear to be relevant
- By computing the Fisher scores (supervised), we have

$$0.02 \quad 0.00 \quad 0.17 \quad 0.67 \quad 1.91 \quad 0.10$$

Features v_4, v_5 are the most relevant for the classification task

Scores based on information theory

Information theory can bring interesting scores for assessing the relevance of the features

- Assuming that the feature values have been discretized, it is possible to estimate the empirical joint and marginal probability distributions of the features and the annotations
- By using the concept of **mutual information**,

$$\text{information_gain}(d) = I(X_{d,:}; \mathbf{y})$$

is a **measure of dependency** between the feature vector $X_{d,:}$ and the annotation vector $\mathbf{y}$. The higher this term is, the more dependent the variables are, and the more useful the feature is.

Scores based on information theory

- When selecting several features, we can extend this criterion by introducing the **Minimum Redundancy Maximum Relevance (MRMR)**, that will both assess the fit between the feature values and the annotations, but also search for uncorrelated features

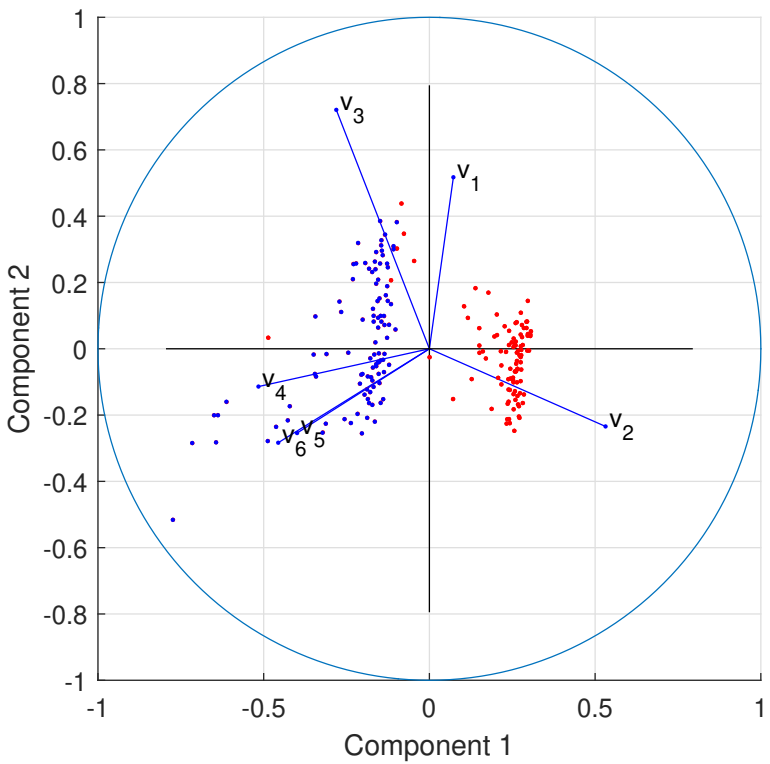
$$\text{MRMR}(d) = I(X_{d,:}; \mathbf{y}) - \frac{1}{|\mathcal{S}|} \sum_{j \in \mathcal{S}} I(X_{d,:}; X_{j,:})$$

  where $\mathcal{S}$ is the current set of selected features. The intuition is that with more non-redundant features selected, it becomes more difficult for new features to be redundant to the features that have already been in $\mathcal{S}$
- Many other criteria exist, for instance based on conditional entropies (see mini-projects)

Wrapper methods

- Test off-the-shelf classifiers (k-NN, SVM, CNN...) with different sets of features and compare the performances obtained with **cross-validation**
- Selection on the set of features giving the **best performances on the training set**
- If D is large, the number of possible configurations is huge ($2^D - 1$). Several strategies can be implemented:
  - **Iterative addition of feature:** we take the best individual feature, then test all groups of two features including that one, etc...
  - **Iterative removal of feature:** we remove the features one-by-one by studying the consequences on the performances
  - **Stochastic approach:** we test random sets of features

Example



- Two classes of signals and $D = 6$ features
- Test with 1-NN (Euclidean distance in the feature space)
- One variable (Classification performances with Leave-One-Out)

$$0.49 \quad 0.97 \quad 0.64 \quad 0.88 \quad 0.94 \quad 0.7850$$

- Two variables (test of all $\binom{6}{2} = 15$ possible configurations)

$$v_2 + v_5 : 1.0$$

$$v_2 + v_3 : 0.99$$

$$v_3 + v_5 : 0.98$$

$$\ldots$$

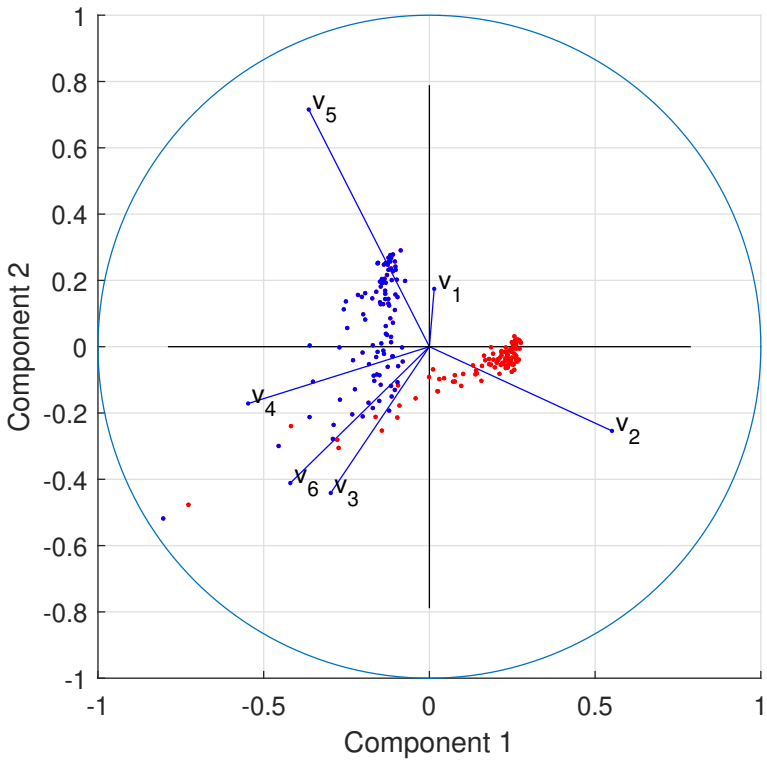
Embedded methods

- Use of regression techniques that induce some sparsity (such as L1-penalized regression or decision trees)
- **LASSO regression (Least Absolute Shrinkage and Selection Operator)**. [Tibshirani, 1996]

$$\mathbf{z}^* = \underset{\mathbf{z}}{\operatorname{argmin}} \left\| \mathbf{y} - \mathbf{X}^\top \mathbf{z} \right\|_2^2 + \lambda \left\| \mathbf{z} \right\|_1$$

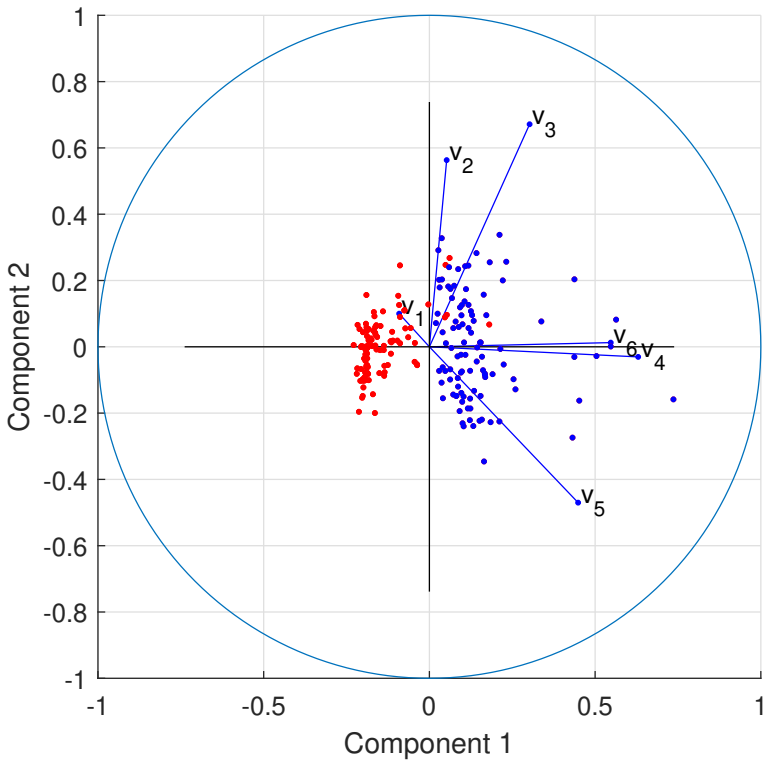
  - Sparse regression to retrieve annotations $\mathbf{y}$ from the data $\mathbf{X}$
  - Coefficients z_d reflect the importance of feature d in the regression. Thanks to the L1 sparsity constraint, lots of coefficients will be set to 0, hence providing a feature selection
  - How to solve the optimization problem: more information on Lecture 3 about Models and Representation Learning (basically proximal gradient descent)

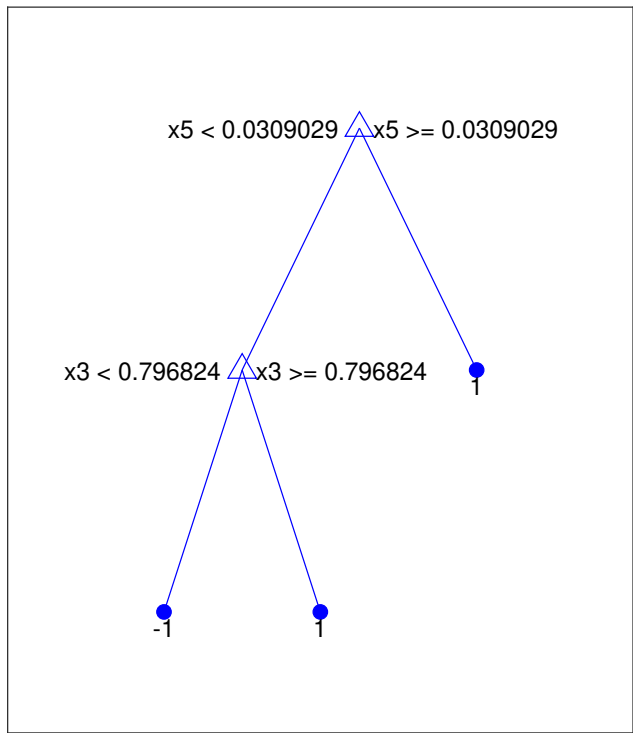
Example: LASSO



- Two classes of signals and $D = 6$ features.
- LASSO algorithm with several values for λ
  - 5 variables (1, 2, 4, 5, 6) : 0.97
  - 4 variables (2, 4, 5, 6) : 0.96
  - 3 variables (2, 5, 6) : 0.95

Example: Decision trees





Possible to only use two variables : 3 and 5

Contents

1. Problem statement

2. Feature Extraction

3. Feature Selection

4. Conclusion

Conclusion

- **Classical features**
  - Statistical, spectral, symbolic and information-theoretic descriptors are well established in the litterature
  - There exist now "universal" feature extractors, so the task has shifted to feature selection
- **Convolutional features**
  - Now regarded as state-of-the-art, particularly in classification, even though their usefulness depends on a supervised selection step
  - Some studies have proposed to systematically explore the *good* random filters to better understand which detected shapes are the most useful
- **Deep learning features**
  - No "AlexNet moment" for time series so far
  - Recurrent neural networks have been supplanted by convolutional neural networks and transformers (but this may still change!)
  - The hot topic right now is model foundations and self-supervised methods
- **Feature selection**
  - These methods, some of which are quite old, are being brought back into fashion
  - Some models (random forests, regularized linear models) perform their own implicit selection and can handle selection naturally

Benchmarks

For **classification**: systematic evaluation over the UCR archive, 112 datasets, 30 resamples [Middlehurst et al., 2024]

Conclusions

- The two leading methods are a **ensemble method** (HIVE-COTE 2.0) and a **random convolution** method (MultiRocket-Hydra)
- Deep learning methods fall into a second group

- **Traditional features are not obsolete**: for extrinsic regression, certain standard processing chains (statistical descriptors + classifier) are among the best methods [Guijo-Rubio et al., 2024]
- **Learned features have not replaced carefully designed features**: expertise remains essential

A few reference papers

- Middlehurst, M., Schäfer, P., & Bagnall, A. (2024). Bake off redux: a review and experimental evaluation of recent time series classification algorithms. Data Mining and Knowledge Discovery, 38(4), 1958-2031. : benchmark for classification
- Guijo-Rubio, D., Middlehurst, M., Arcencio, G., Silva, D. F., & Bagnall, A. (2024). Unsupervised feature based algorithms for time series extrinsic regression. Data Mining and Knowledge Discovery, 38, 2141-2185. : benchmark for extrinsic regression
- Lubba, C. H., Sethi, S. S., Knaute, P., Schultz, S. R., Fulcher, B. D., & Jones, N. S. (2019). catch22: canonical time-series characteristics. Data Mining and Knowledge Discovery, 33(6), 1821-1852. : canonical feature set
- Fulcher, B. D., & Jones, N. S. (2017). hctsa: a computational framework for automated time-series phenotyping using massive feature extraction. Cell Systems, 5(5), 527-531. : massive feature extraction
- Dempster, A., Petitjean, F., & Webb, G. I. (2020). ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels. Data Mining and Knowledge Discovery, 34(5), 1454-1495. : random convolutional features
- Yue, Z., Wang, Y., Duan, J., Yang, T., Huang, C., Tong, Y., & Xu, B. (2022). TS2Vec: towards universal representation of time series. In Proceedings of the AAAI Conference on Artificial Intelligence (pp. 8980-8987). : self-supervised representation learning
- Li, J., Cheng, K., Wang, S., Morstatter, F., Trevino, R. P., Tang, J., & Liu, H. (2017). Feature selection: a data perspective. ACM Computing Surveys, 50(6), 1-45. : survey on feature selection

Code and resources

Feature extraction

- `pycatch22` - https://github.com/DynamicsAndNeuralSystems/pycatch22
  The 22 canonical features, computable in linear time
- `tsfresh` - https://tsfresh.readthedocs.io
  Around 800 features, with built-in selection by hypothesis testing
- `TSFEL` - https://tsfel.readthedocs.io
  Statistical, temporal and spectral features
- `aeon` - https://www.aeon-toolkit.org
  ROCKET and its variants, symbolic transforms (SAX, SFA), shapelet transforms

Feature selection

- `scikit-learn` - https://scikit-learn.org
  PCA, LASSO, mutual information, recursive feature elimination, variance thresholding
- `scikit-feature` - https://jundongl.github.io/scikit-feature
  The implementations accompanying the survey of Li et al.

References

- Pincus, S. M. (1991). Approximate entropy as a measure of system complexity. Proceedings of the National Academy of Sciences, 88(6), 2297-2301.
- Schäfer, P. (2015). The BOSS is concerned with time series classification in the presence of noise. Data Mining and Knowledge Discovery, 29(6), 1505-1530.
- Lin, J., Keogh, E., Wei, L., & Lonardi, S. (2007). Experiencing SAX: a novel symbolic representation of time series. Data Mining and knowledge discovery, 15(2), 107-144.
- Chung, Y. A., Wu, C. C., Shen, C. H., Lee, H. Y., & Lee, L. S. (2016). Audio word2vec: Unsupervised learning of audio segment representations using sequence-to-sequence autoencoder. arXiv preprint arXiv:1603.00982.
- Le Guen, V., & Thome, N. (2019). Shape and time distortion loss for training deep time series forecasting models. Advances in neural information processing systems, 32..
- Gers, F. A., Schmidhuber, J., & Cummins, F. (1999). Learning to forget: Continual prediction with LSTM.
- Wold, S., Esbensen, K., & Geladi, P. (1987). Principal component analysis. Chemometrics and intelligent laboratory systems, 2(1-3), 37-52.
- Song, F., Guo, Z., & Mei, D. (2010, November). Feature selection using principal component analysis. In 2010 international conference on system science, engineering design and manufacturing informatization (Vol. 1, pp. 27-30). IEEE.
- Li, J., Cheng, K., Wang, S., Morstatter, F., Trevino, R. P., Tang, J., & Liu, H. (2017). Feature selection: A data perspective. ACM Computing Surveys (CSUR), 50(6), 1-45.
- Gu, Q., Li, Z., & Han, J. (2011, July). Generalized Fisher score for feature selection. In Proceedings of the Twenty-Seventh Conference on Uncertainty in Artificial Intelligence (pp. 266-273).
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. Journal of the Royal Statistical Society: Series B (Methodological), 58(1), 267-288.

Lecture 2: list of possible topics/projects (1/2)

Symbolic features

Project 2.1 Schäfer, P. (2015). The BOSS is concerned with time series classification in the presence of noise. Data Mining and Knowledge Discovery, 29(6), 1505-1530.
How to use local symbolic features to classify time series

Project 2.2 Le Nguyen, T., Gsponer, S., Ilie, I., O'Reilly, M., & Ifrim, G. (2019). Interpretable time series classification using linear models and multi-resolution multi-domain symbolic representations. Data Mining and Knowledge Discovery, 33(4), 1183-1222.
How to use multiscale symbolic representations to classify time series

Project 2.3 Schäfer, P., & Leser, U. (2023). WEASEL 2.0: a random dilated dictionary transform for fast, accurate and memory constrained time series classification. Machine Learning, 112, 4763-4788.
How to bring dilation and randomization into symbolic representations

Information theory features

Project 2.4 Rostaghi, M., & Azami, H. (2016). Dispersion entropy: a measure for time-series analysis. IEEE Signal Processing Letters, 23(5), 610-614.
How to measure complexity without the tolerance parameter of approximate entropy

Project 2.5 Richman, J. S., & Moorman, J. R. (2000). Physiological time-series analysis using approximate entropy and sample entropy. American Journal of Physiology: Heart and Circulatory Physiology, 278(6), H2039-H2049.
How to correct the self-matching bias of approximate entropy on physiological time series

Convolutional features

Project 2.6 Dempster, A., Schmidt, D. F., & Webb, G. I. (2021, August). MiniRocket: a very fast (almost) deterministic transform for time series classification. In Proceedings of the 27th ACM Conference on Knowledge Discovery and Data Mining (pp. 248-257).
How to remove almost all the randomness of the filters and gain two orders of magnitude

Project 2.7 Tan, C. W., Dempster, A., Bergmeir, C., & Webb, G. I. (2022). MultiRocket: multiple pooling operators and transformations for fast and effective time series classification. Data Mining and Knowledge Discovery, 36, 1623-1646.
How to enrich the features extracted from a single activation map

Project 2.8 Dempster, A., Schmidt, D. F., & Webb, G. I. (2023). Hydra: competing convolutional kernels for fast and accurate time series classification. Data Mining and Knowledge Discovery, 37, 1779-1805.
How to make filters compete, and recover a dictionary of shapes

Lecture 2: list of possible topics/projects (2/2)

Deep learning features

Project 2.9 Franceschi, J. Y., Dieuleveut, A., & Jaggi, M. (2019). Unsupervised scalable representation learning for multivariate time series. Advances in Neural Information Processing Systems, 32.
How to learn features without labels using a triplet loss

Project 2.10 Yue, Z., Wang, Y., Duan, J., Yang, T., Huang, C., Tong, Y., & Xu, B. (2022, June). TS2Vec: towards universal representation of time series. In Proceedings of the AAAI Conference on Artificial Intelligence (pp. 8980-8987).
How to learn features usable at any granularity through hierarchical contrastive learning

Project 2.11 Ismail Fawaz, H., Lucas, B., Forestier, G., Pelletier, C., Schmidt, D. F., Weber, J., ... & Petitjean, F. (2020). InceptionTime: finding AlexNet for time series classification. Data Mining and Knowledge Discovery, 34(6), 1936-1962.
How to design a multiscale deep architecture and extract features from it

Topological features

Project 2.12 Umeda, Y. (2017). Time series classification via topological data analysis. Transactions of the Japanese Society for Artificial Intelligence, 32(3), D-G72_1-12.
How to turn the topology of a time series into a curve usable as a feature

Project 2.13 Bois, A., Tervil, B., Moreau, A., Vienne-Jumeau, A., Ricard, D., & Oudre, L. (2022). A topological data analysis-based method for gait signals with an application to the study of multiple sclerosis. PLOS ONE, 17(5), e0268475.
How to extract topological features from gait signals to study a neurological disease

Feature selection

Project 2.14 Lindenbaum, O., Shaham, U., Peterfreund, E., Svirsky, J., Casey, N., & Kluger, Y. (2021). Differentiable unsupervised feature selection based on a gated Laplacian. Advances in Neural Information Processing Systems, 34, 1530-1542.
How to perform unsupervised feature selection with a differentiable gating mechanism

Project 2.15 Christ, M., Braun, N., Neuffer, J., & Kempa-Liehr, A. W. (2018). Time series feature extraction on basis of scalable hypothesis tests. Neurocomputing, 307, 72-77.
How to combine massive feature extraction with selection based on statistical tests

Project 2.16 Lubba, C. H., Sethi, S. S., Knaute, P., Schultz, S. R., Fulcher, B. D., & Jones, N. S. (2019). catch22: canonical time-series characteristics. Data Mining and Knowledge Discovery, 33(6), 1821-1852.
How to reduce thousands of candidate features to a canonical set of twenty-two

Project 2.17 Li, J., Cheng, K., Wang, S., Morstatter, F., Trevino, R. P., Tang, J., & Liu, H. (2017). Feature selection: a data perspective. ACM Computing Surveys, 50(6), 1-45.
How to apply a large number of feature selection methods (two groups can take this)

Project 2.18 Guijo-Rubio, D., Middlehurst, M., Arcencio, G., Silva, D. F., & Bagnall, A. (2024). Unsupervised feature based algorithms for time series extrinsic regression. Data Mining and Knowledge Discovery, 38, 2141-2185.
How to build a complete feature-based pipeline, and whether selecting features actually helps